

SOFTWARE ENGINEERING

02

Introduction to Web Technologies – HTML & CSS

From blank screens to beautiful, responsive web pages — one tag at a time.

DURATION	LEVEL	MODULE	UPDATED
12 Hours · 12 Sessions + 1 Mini Project	Beginner	Module 2	May 2026

IN THIS MODULE

01	Introduction to Web Design
02	HTML Page Structure
03	Forms & Inputs
04	Semantic HTML5
05	Introduction to CSS3
06	Box Model & Positioning
07	Flexbox & Grid
08	Responsive Design
09	Bootstrap Basics
10	Bootstrap Components
11	Bootstrap Utilities & Customisation
12	Integration Review
MP	Mini Project – Social Bio Link Page

SESSION

01

60 MIN · FUNDAMENTALS

Introduction to Web Design

◆ Learning Objectives

- Explain how a browser turns HTML and CSS files into the page you see on screen
- Distinguish between front-end and back-end responsibilities in a web project
- Identify the roles of HTML, CSS, and JavaScript in building a webpage
- Describe what the DOM is and why developers need to understand it
- Open Chrome DevTools and inspect elements on any live website

01 What Is a Website, Really?

When you open Swiggy on your phone in Ahmedabad, you see food images, prices, and buttons — but your device received only plain text files from Swiggy's server. The browser read those files and painted everything you see on screen. That process, from raw text to a living page, is exactly what this module is about.

CONCEPT — Three Languages of the Web

Every webpage is built with three layers. HTML provides the structure — headings, paragraphs, buttons. CSS controls the look — colours, fonts, spacing. JavaScript adds behaviour — animations, form checks, live updates. Think of HTML as the walls of a shop in Lal Darwaja market, CSS as the interior design, and JavaScript as the staff who interact with customers.

Front-End

Everything the user sees and interacts with — buttons, forms, colours, layout. Built with HTML, CSS, and JavaScript. Runs inside the user's browser. Examples: Flipkart's product page, IRCTC's booking form.

Back-End

Everything that happens on the server — processing payments, storing user data, running business logic. Built with Python, PHP, Java, or Node.js. The user never sees this code directly.

02 How Browsers Render a Page

When you hit Enter after typing a URL, the browser fetches the HTML file and reads it top to bottom, building a tree of all the page's elements in memory. It then fetches any linked CSS files and applies styles to each element before finally painting pixels on your screen. This entire sequence — from your keypress to visible content — typically completes in under a second.

CONCEPT — The DOM (Document Object Model)

The DOM is the browser's live, in-memory representation of your HTML page as a tree of nodes. Each tag — body, h1, p — becomes a node in that tree. JavaScript can read and change any node, which is why clicking 'Add to Cart' on Amazon updates the page without reloading. You can inspect the DOM of any page at any time using Chrome DevTools.

"The browser is your compiler. HTML is your source code."

03 Chrome DevTools — Your First Superpower

Every professional web developer has Chrome DevTools open alongside their code. It lets you inspect any element on any website, see its exact CSS, watch network requests happen in real time, and test changes live — without editing a single file. You will use DevTools in every session of this module.

TRY THIS LIVE

Open any website — try [instagram.com](https://www.instagram.com) or [flipkart.com](https://www.flipkart.com). Press F12. Click the cursor icon in the top-left corner of DevTools. Now hover over the page — hover over the search bar, a product image, a button. Click on any element. Watch the Elements panel on the left highlight the exact HTML tag behind what you clicked.

F12	Open / close Chrome DevTools
Ctrl + Shift + C	Activate the Inspect Element picker
Ctrl + Shift + I	Jump to the Elements panel directly
Ctrl + Shift + J	Jump to the Console panel directly
Ctrl + R	Hard-reload and clear cache

04 The Scale of What You Are Learning

INDIA'S ACTIVE INTERNET USERS

900M+

Source: TRAI Annual Report, 2024 — every one of them uses websites built with HTML, CSS, and JavaScript.

CASE STUDY — Zepto (Mumbai -> pan-India)

Context — Zepto launched in 2021 as a 10-minute grocery delivery startup in Mumbai and expanded rapidly across Ahmedabad, Surat, and other Tier-1 Indian cities.

Challenge — The team needed a storefront that loaded extremely fast on 4G and low-end Android phones, while displaying real-time stock and pricing for thousands of SKUs.

What they did — Front-end engineers used minimal, semantic HTML to keep page weight low, CSS for a clean mobile layout, and JavaScript only for dynamic elements like the cart counter and live inventory — exactly the three-layer model introduced in this session.

Result — Zepto achieved a PageSpeed mobile score above 80, well above the e-commerce average, directly contributing to a low-bounce checkout experience at scale.

Takeaway — The front-end / back-end split and a disciplined use of HTML, CSS, and JavaScript are business decisions, not just technical ones.

Source: Zepto engineering blog and web.dev case studies (2022–2023)

WATCH — VIDEO REFERENCE (HINDI / ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Internet Technologies — Lecture 1: Introduction to the Web (IIT Kharagpur). Trainer: verify URL at nptel.ac.in before class.

Link — <https://nptel.ac.in/courses/106105084>

Always link the official channel — beware fan-made or re-upload copies.

WHERE TO GO NEXT

You now understand what browsers do and how to inspect any live page — that mental model underpins every session in this module.

Explore deeper — Read 'How Browsers Work' by Tali Garsiel (search the title — it is freely available and considered a must-read for front-end developers).

Try on your own — Open three Indian websites — Zepto, MakeMyTrip, and NPCI's UPI page. Use DevTools to find the main nav element on each. See how different companies structure the same component.

Connect the dots — In Session 2 you write your first HTML file — now you know exactly where that file lives in the browser's pipeline.

Going further — Look up the browser Rendering Pipeline: Parse -> Style -> Layout -> Paint -> Composite. Each stage is a separate engineering discipline with tools and specialists.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a web development teacher. Explain the difference between front-end and back-end development in

simple Hinglish, using the example of ordering food on Zomato. Cover what the user sees, what happens on the server, and which technologies power each side. Keep it under 150 words.

SESSION

02

60 MIN · HTML BASICS

HTML Page Structure

◆ Learning Objectives

- Write a valid HTML5 boilerplate from memory
- Apply heading tags h1–h6 with a correct content hierarchy
- Add paragraphs, ordered lists, and unordered lists to a page
- Insert hyperlinks using the anchor tag with correct href and target attributes
- Embed images with the img tag and a meaningful alt attribute
- Build a basic HTML table with header and data rows
- Combine all of the above into a complete 'About Me' HTML page

01 The HTML Boilerplate

Every HTML file in the world starts with the same skeleton. This boilerplate tells the browser which version of HTML you are using, sets the character encoding so Gujarati and Hindi characters display correctly, and divides the file into a head section (invisible settings) and a body section (everything the user actually sees).

<code><!DOCTYPE html></code>	Tells the browser: this is HTML5
<code><html lang="en"></code>	Root element; lang declares the page language
<code><head></code>	Invisible section — metadata, title, CSS links go here
<code><meta charset="UTF-8"></code>	Allows Gujarati, Hindi and all Unicode characters to render
<code><meta name="viewport" content="width=device-width, initial-scale=1.0"></code>	Makes the page display correctly on mobile screens
<code><title>My First Page</title></code>	Text shown on the browser tab
<code></head></code>	End of head section
<code><body></code>	Everything visible to the user goes inside here
<code><h1>Hello, TOPS Technologies!</h1></code>	Your first visible element
<code></body></code>	End of body section
<code></html></code>	Close the root element — always the last line

PRO TIP — VS Code Emmet Shortcut

In VS Code, create a new file called index.html, type a single ! (exclamation mark), and press Tab or Enter. The full HTML5 boilerplate appears instantly. This is Emmet — it is built into VS Code and works for dozens of HTML patterns. You will never need to type the boilerplate by hand again.

02 Text Tags — Headings, Paragraphs & Lists

HTML gives you six heading levels — h1 to h6 — to organise content like an outline. Use h1 for the page title once per page. Use h2 for major sections and h3 for sub-sections beneath them. Think of it like a newspaper: the masthead ('Ahmedabad Mirror') is h1, a story headline is h2, and a photo caption is h3. Never skip a level — jump from h1 straight to h3 and screen readers lose track of your structure.

<code><h1>Page Title – once per page</h1></code>	Largest heading; also the most important for SEO
<code><h2>Section Heading</h2></code>	Major sections of the page
<code><h3>Sub-Section</h3></code>	Content nested under an h2
<code><p>This is a paragraph.</p></code>	Block of text; browser adds top/bottom spacing automatically
<code>Bold text</code>	Semantic bold — signals importance to search engines
<code>Italic text</code>	Semantic italic — signals emphasis
<code>
</code>	Single line break inside a paragraph (use sparingly)

◆ Ordered & Unordered Lists

<code></code>	Unordered list — renders as bullet points
<code>HTML</code>	List item — must be a direct child of ul or ol
<code>CSS</code>	Another item
<code></code>	Close the unordered list
<code></code>	Ordered list — renders as 1, 2, 3... (use for steps)
<code>Open VS Code</code>	Step 1
<code>Create index.html</code>	Step 2
<code>Type ! and press Tab</code>	Step 3
<code></code>	Close the ordered list

CONCEPT — Block vs Inline Elements

Block elements (h1–h6, p, ul, ol, li) take up the full width of the page and always start on a new line — like paragraphs in a Word document. Inline elements (strong, em, a, img) sit inside a line of text without breaking it — like bold or underlined words mid-sentence. This distinction is the foundation of CSS layout in Sessions 5 and 6.

03 Links, Images & Tables

The anchor tag is what makes the web a web — without it, every page would be an island with no connection to any other. The href attribute holds the destination URL. Adding target="_blank" opens the link in a new tab, which is standard practice for external sites so users do not lose their place on your page.

```
<a href="https://www.tops-int.com">Visit TOPS</a>
```

Basic hyperlink

```
<a href="https://www.tops-int.com" target="_blank">Open in new tab</a>
```

Opens without navigating away from current page

```
<a href="#contact">Jump to Contact</a>
```

In-page anchor — links to an element with id='contact'

```

```

Embed a local image; alt is required for accessibility

```

```

Embed an image from a URL

WARNING — Never skip the alt attribute

The alt attribute on every img tag is not optional — it is required by accessibility standards (WCAG) and by Google's indexing bots. Screen readers read it aloud for visually impaired users. Write a short description of what the image shows: alt="Profile photo of Priya Shah" not alt="image1".

◆ HTML Tables

<code><table></code>	Outer container for the entire table
<code><tr></code>	Table row — wraps all cells in one row
<code><th>Name</th></code>	Table header cell — bold and centred by default
<code><th>Course</th></code>	Another header cell
<code></tr></code>	End of the header row
<code><tr></code>	First data row
<code><td>Priya Shah</td></code>	Table data cell
<code><td>Web Dev</td></code>	Another data cell
<code></tr></code>	End of the data row
<code></table></code>	Close the table

TRY THIS LIVE — Build Your About Me Page

Open VS Code -> create about.html -> type ! + Tab for the boilerplate. Now add: (1) an h1 with your full name, (2) an h2 'About Me' followed by a short paragraph, (3) an unordered list of 3 skills you want to learn, (4) an anchor linking to tops-int.com that opens in a new tab, (5) an img tag pointing to any image file, (6) a table with two columns — 'Course' and 'Duration' — listing two TOPS courses. Save and open in Chrome. Use DevTools to inspect each element you added.

04 Common Mistakes to Avoid

Mistake	What Goes Wrong	Fix
Forgetting to close a tag	Everything after it may be misformatted or invisible	Use VS Code's auto-close and check the Elements panel in DevTools
Using h3 without an h2 above it	Breaks accessibility and confuses search engine crawlers	Always use headings in strict order: h1 -> h2 -> h3
Missing alt on img	Fails accessibility (WCAG) and Google image indexing	Always write a descriptive alt attribute for every image
Using br tags to create spacing	Fragile layout that breaks on different screen sizes	Use CSS margin or padding instead — covered in Session 6
Wrong file path in src or href	Broken image or 404 link	Paths are case-sensitive on Linux servers; double-check spelling and slashes

PRO TIP — Validate Your HTML

Paste your HTML into validator.w3.org — the W3C's free HTML validator. It catches unclosed tags, missing attributes, and structural errors before your browser silently 'fixes' them in ways you did not intend. Make validation a habit after finishing every new page.

WATCH — VIDEO REFERENCE (HINDI / ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Internet Technologies — Lecture 2: HTML Basics (IIT Kharagpur). Trainer: verify URL at nptel.ac.in before class.

Link — <https://nptel.ac.in/courses/106105084>

WHERE TO GO NEXT

You can now write a structurally valid HTML5 page from scratch — the foundation that every CSS, JavaScript, and Bootstrap session builds directly on top of.

Explore deeper — Bookmark the MDN HTML Element reference at developer.mozilla.org/en-US/docs/Web/HTML/Element — it is the most complete and accurate HTML reference available and you will return to it throughout this course.

Try on your own — Recreate any one section of the TOPS Technologies homepage (tops-int.com) using only HTML — no CSS. Focus purely on structure: headings, paragraphs, lists, images, and links. See how much content you can capture before styling.

Connect the dots — Session 3 adds forms — text inputs, dropdowns, checkboxes, and buttons — which are what make HTML pages interactive and capable of collecting user data.

Going further — Read about HTML5 semantic elements — header, main, article, section, aside, footer. They replace generic div tags with meaningful structure and are the focus of Session 4.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as an HTML teacher. Explain the difference between block-level elements and inline elements in simple Hinglish, using an analogy from everyday Indian life. Give two examples of each type and explain when a developer would choose one over the other. Keep it under 150 words.

SESSION

03

60 MIN · HTML FORMS

Forms & Inputs

◆ Learning Objectives

- Explain how HTML forms collect and transmit data to a server
- Choose correctly between the GET and POST form methods
- Build a form with text, email, password, number, checkbox, and radio inputs
- Use label tags to associate descriptions with inputs accessibly
- Add a select dropdown and a textarea field to a form
- Apply HTML5 validation attributes to prevent invalid submissions
- Inspect form-submitted data in Chrome DevTools' Network tab

01 Why Forms Are the Heart of the Web

Without forms, the web would be a read-only library. Every time a student registers on Naukri.com, books a train ticket on IRCTC, or submits a Aadhaar self-update request, an HTML form is handling that action. Forms are the primary mechanism through which users give data to a web application — and as a front-end developer, you will build them in nearly every project.

CONCEPT — How Forms Work

A form collects input from the user and sends it to a server when submitted. The form tag has two key attributes: action (the URL the data is sent to) and method (how it travels). GET appends data to the URL as query strings — used for search and filtering. POST sends data hidden in the request body — used for login, registration, and any sensitive submission.

IRCTC DAILY TICKET BOOKINGS

8 Lakh+

Source: IRCTC Annual Report, 2023 — every single booking begins with an HTML form.

02 The form Tag & Input Types

All input elements must live inside a form tag to be submitted together as one package. The input tag is a void element — it has no closing tag — and its type attribute controls what kind of data it collects. HTML5 offers over 20 input types; the seven below cover 90% of real-world forms.

```
<form action='/register' method='POST'>
  <input type='text' name='username' placeholder='Name'>
    Single-line text — names, cities, usernames
  <input type='email' name='email'>
    Email — browser validates @ and domain
    automatically
  <input type='password' name='password'>
    Hides characters as the user types
  <input type='number' name='age' min='16' max='60'>
    Numeric input with optional min and max bounds
  <input type='tel' name='phone' placeholder='9876543210'>
    Phone — triggers a numeric keypad on mobile
  <input type='checkbox' name='agree' id='agree'>
    Checkbox — value is 'on' when checked, absent otherwise
  <input type='radio' name='gender' value='male'> Male
    Radio — same name groups them so only one can be selected
  <input type='submit' value='Register'>
    Submits all inputs above to the action URL
</form>
    All inputs inside are submitted together as one request
```

PRO TIP — name vs id on Input Elements

The name attribute is what the server reads when form data arrives — it is the key in the key-value pair the server receives. The id attribute is what CSS and JavaScript use to target the element in the browser. Both are needed, but they serve completely different audiences. A form submitted without name attributes on its inputs will arrive at the server with no data at all.

03 Labels, Select & Textarea

A label tag links a text description to its input field using matching for and id attributes. When done correctly, clicking the label text focuses the input — making forms easier to use on mobile touchscreens. Screen readers for visually impaired users also depend on labels to announce what each field is for. Labels are an accessibility requirement, not optional polish.

```
<!-- Linking a label to an input -->
<label for='email'>Email Address</label>
  for must match the input's id exactly — case-sensitive
<input type='email' id='email' name='email' required>
  id='email' links this input to the label above
<!-- Dropdown select -->
<select name='course'>
  Dropdown menu — the selected option's value goes to
  the server
  <option value=''>-- Select a Course --</option>
    Placeholder option — empty value, shown first
  <option value='webdev'>Web Development</option>
    Selectable option — value is sent, not the visible text
```

```
<option value='python'>Python Programming</option>
  Another option
</select>                                Close the dropdown
<!-- Multi-line text -->
<textarea name='bio' rows='4' placeholder='Tell us about yourself...'></textarea>
  Multi-line text; rows sets initial height; always needs a closing tag
```

TRY THIS LIVE — Social Bio Link Registration Form

Build the registration form for this module's Mini Project. Create a new HTML file and add: a text input for full name (with label), an email input, a password input, a select dropdown listing Instagram, LinkedIn, and YouTube, a textarea for a short bio, and a submit button. Use `action='#'` so the page submits to itself. Style will come in Session 5 — focus on structure now.

04 Validation & Inspecting Submissions

Before HTML5, developers wrote JavaScript to validate every field manually. Now browsers do first-level validation for free using a small set of attributes on the input tag. The browser blocks submission and shows a native error message automatically — no JavaScript needed for basic checks.

```
<input type='text' name='username' required minlength='3' maxlength='20'>
  Required; must be 3–20 characters
<input type='email' name='email' required>           type='email' validates @ and domain format
                                                       automatically
<input type='tel' name='phone' pattern='[6-9][0-9]{9}' title='10-digit mobile number'>
  Regex for valid Indian mobile numbers (must start 6–9)
<input type='number' name='age' min='16' max='60' step='1'>
  Must be a whole number between 16 and 60
<input type='text' name='pincode' pattern='[1-9][0-9]{5}' required>
  6-digit Indian PIN code — first digit cannot be 0
```

Attribute	What It Does	Example
required	Field must be filled; blocks submission if empty	required
minlength	Minimum number of characters the user must enter	minlength='8'
maxlength	Maximum number of characters allowed	maxlength='20'

Attribute	What It Does	Example
pattern	Value must match this regular expression	pattern='[A-Za-z]+'
min / max	Numeric or date boundary for number and date inputs	min='1' max='100'
type='email'	Auto-validates presence of @ and a domain suffix	type='email'
type='number'	Restricts to numbers; enables min, max, and step	type='number' step='0.01'

WARNING — HTML5 Validation Is Not Security

HTML5 validation runs only inside the browser. Anyone with basic technical knowledge can bypass it by disabling JavaScript or sending a raw HTTP request. Always validate data again on the server. Think of client-side validation as convenience for the user and server-side validation as security for the application — you need both.

TRY THIS LIVE — Watch Your Form Data in DevTools

Set your form's method to GET and action to '#'. Open DevTools -> Network tab -> enable 'Preserve log'. Fill in the form and submit. Click the request that appears. Under the Headers tab, find 'Query String Parameters' — every field name and value your form sent is listed there. This is exactly what a server sees when your form is submitted.

WATCH — VIDEO REFERENCE (HINDI / ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Internet Technologies — lecture on HTML Forms and Interaction (IIT Kharagpur). Trainer: verify lecture number at nptel.ac.in before class.

Link — <https://nptel.ac.in/courses/106105084>

WHERE TO GO NEXT

You can now build fully structured, validated HTML forms — the data-collection layer that every web application depends on.

Explore deeper — Read MDN's guide 'Your first HTML form' at developer.mozilla.org/en-US/docs/Learn/Forms/Your_first_form — it covers the complete request lifecycle in detail.

Try on your own — Open any Indian government form (uidai.gov.in, passportindia.gov.in) and list every

input type, label, and validation pattern you can spot using DevTools.

Connect the dots — Session 4 introduces semantic HTML5 — including the fieldset and legend elements that give form sections proper structural meaning for screen readers.

Going further — Look up the Constraint Validation API — it lets JavaScript read, customise, and trigger HTML5 validation messages programmatically for production-grade forms.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a web development teacher. Explain the difference between GET and POST form methods in simple Hinglish, using a search bar (GET) and a login form (POST) as examples. Explain why using POST for passwords matters. Keep it under 150 words.

SESSION

04

60 MIN · HTML STRUCTURE

Semantic HTML5

◆ Learning Objectives

- Explain what 'semantic' means in the context of HTML and why it matters
- Use header, nav, main, article, section, aside, and footer tags correctly
- Replace non-descriptive div containers with the appropriate semantic tag
- Build a YouTube-style page layout using only semantic HTML5 elements
- Describe how semantic structure improves accessibility and search engine ranking

01 What's Wrong with Using Only Divs?

Before HTML5, developers built every page layout with div tags stacked inside div tags. A navigation was a div. The main content area was a div. The footer was a div. This 'div soup' worked visually — browsers displayed it fine — but it carried no meaning. A screen reader seeing 50 divs has no idea which one is the navigation and which is the article. Neither does Google's crawler.

Div Soup (Pre-HTML5)

Every part of the page is a div with a class name: `div.header`, `div.nav-bar`, `div.main-content`, `div.sidebar`, `div.footer`. The class names only mean something to the developer who wrote them. Browsers, screen readers, and search engines see only generic boxes with no role or purpose.

Semantic HTML5

Each tag declares its own purpose: `header`, `nav`, `main`, `article`, `aside`, `footer`. Browsers expose these roles to accessibility tools. Screen readers let users jump between sections by keyboard. Google's crawler understands page structure and weights content accordingly — better SEO, no extra effort.

CONCEPT — What 'Semantic' Means

A semantic element is one whose tag name communicates its meaning — both to the browser and to any developer reading the code. `div` and `span` are non-semantic: they are empty containers with no implied purpose. `header`, `nav`, `main`, `article`, `section`, `aside`, and `footer` are semantic: each tag tells you exactly what role that block plays in the page.

02 The Semantic Tag Family

HTML5 introduced seven structural semantic tags that cover the layout of virtually every web page. Learn these seven and you can scaffold any site correctly on the first attempt. Each tag can also appear multiple times on a page — for example, a page can have a main header and individual article headers, and both use the `header` tag correctly.

Tag

Purpose

Replaces

Tag	Purpose	Replaces
header	Logo, site title, and top-level navigation for a page or section	div class='header'
nav	The primary navigation links — menus, breadcrumbs, pagination	div class='nav'
main	The dominant, unique content of the page — use only once per page	div class='main-content'
article	Self-contained content that makes sense on its own: blog post, product card	div class='post'
section	A thematic grouping of related content inside main or article	div class='section'
aside	Content related to but secondary to the main flow: sidebars, related links	div class='sidebar'
footer	Page or section footer: copyright notice, contact links, site map	div class='footer'

<code><body></code>	Page root
<code> <header></code>	Site header — logo, nav
<code> </code>	
Brand logo	
<code> <nav></code>	Primary navigation
<code> Home</code>	Nav link
<code> Courses</code>	Nav link
<code> </nav></code>	Close nav
<code></header></code>	Close header
<code><main></code>	Primary content — used once per page
<code> <section class='video-grid'></code>	Thematic group of video cards
<code><article class='video-card'>...</article></code>	
One self-contained video item	
<code> </section></code>	Close section
<code> <aside></code>	Sidebar — recommendations, ads
<code> <h3>Recommended</h3></code>	Sidebar heading
<code> </aside></code>	Close aside
<code></main></code>	Close main
<code><footer></code>	Site footer
<code><p>(c) 2025 TOPS Technologies. All rights reserved.</p></code>	
Copyright notice	
<code></footer></code>	Close footer
<code></body></code>	Close body

03 Demo — Build a YouTube-Style Layout

YouTube's page structure is a textbook example of semantic HTML in action: a fixed header with the logo and search nav, a main content area holding a grid of video article cards, a sidebar aside with recommendations, and a footer. Build a simplified version of this layout — the goal is correct structure, not visual styling. CSS comes in Session 5.

```
<header>                                Fixed top bar
  <img src='logo.png' alt='YourTube'>    Logo
  <nav>                                    Top navigation
    <a href='/'>Home</a> <a href='/trending'>Trending</a>
    Navigation links
  </nav>
</header>
<main>                                    All primary content
  <section class='video-grid'>           Grid of video cards
    <article class='video-card'>        One self-contained video card
      <img src='thumb.jpg' alt='HTML Crash Course thumbnail'>
      Thumbnail — alt required
      <h2>HTML Crash Course - Session 1</h2> Video title
      <p>TOPS Technologies · 12K views · 2 days ago</p>
      Metadata
    </article>                          Repeat article for each video
  </section>
  <aside>                                Recommendations sidebar
    <h3>Recommended</h3>
    <ul><li><a href='#'>JavaScript Basics</a></li></ul>
    Recommended link list
  </aside>
</main>
<footer><p>(c) 2025 YourTube</p></footer> Site footer
```

TRY THIS LIVE — Semantic Page Rebuild

Take the 'About Me' page you built in Session 2 and restructure it using semantic tags. Wrap your name and the nav links in a header. Move your bio paragraph and skills list into a main tag with a section inside. Add an aside with 'Quick Links' to your social profiles. Wrap the bottom copyright line in a footer. Inspect the result in DevTools — notice how the Elements panel now reads like an outline of the page.

04 Why This Matters Beyond the Code

TOP WEBSITES FAILING ACCESSIBILITY

96.3%

Source: WebAIM Million Report, 2024 — poor HTML structure is one of the primary causes. Semantic HTML is the first fix.

CASE STUDY — National Informatics Centre (NIC), India

Context — India's National Informatics Centre manages 10,000+ government websites including india.gov.in, uidai.gov.in, and irctc.co.in, serving hundreds of millions of citizens.

Challenge — Government websites were built with div-heavy, non-semantic HTML, making them inaccessible to India's estimated 26 million visually impaired citizens who rely on screen readers.

What they did — NIC mandated semantic HTML5 compliance across all government portals through the Guidelines for Indian Government Websites (GIGW), requiring proper use of header, nav, main, article, footer, and ARIA landmark roles.

Result — Screen reader compatibility improved significantly across government portals, and the portals now meet the Ministry of Electronics and IT's accessibility mandate — a legal requirement under the Rights of Persons with Disabilities Act, 2016.

Takeaway — Semantic HTML is not just good practice for private companies — in India, it is a legal compliance requirement for any public digital service.

Source: NIC GIGW Guidelines v2.0, 2021 (gigw.in) and MeitY Accessibility Policy

PRO TIP — Validate Your Page Structure

Install the WAVE Web Accessibility Evaluation Tool (browser extension by WebAIM) and run it on any page you build. It flags missing labels, skipped heading levels, missing alt text, and improper landmark structure in real time. Running WAVE on your pages before submission is a habit every professional developer develops early.

WATCH — VIDEO REFERENCE (HINDI / ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Internet Technologies — lecture on HTML5 Structural Elements (IIT Kharagpur). Trainer: verify lecture number at nptel.ac.in before class.

Link — <https://nptel.ac.in/courses/106105084>

WHERE TO GO NEXT

You now write HTML that means something — to browsers, to screen readers, to search engines, and to every developer who reads your code after you.

Explore deeper — Read the W3C WAI tutorial on 'Page Structure' at [w3.org/WAI/tutorials/page-structure/](https://www.w3.org/WAI/tutorials/page-structure/) — it is the definitive reference for semantic landmark regions.

Try on your own — Open three popular Indian websites — try Zomato, Zerodha, and Paytm — and use DevTools to check which semantic tags each site actually uses in their layout. See which comes closest to best practice.

Connect the dots — Session 5 applies CSS to your semantic structure — you will see how targeting header, nav, article, and aside with CSS is cleaner and more maintainable than targeting divs by class name.

Going further — Look up ARIA (Accessible Rich Internet Applications) roles — they extend semantic meaning to interactive components like modals, tooltips, and custom dropdowns that have no native HTML5 tag.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a web accessibility teacher. Explain the difference between semantic HTML and non-semantic HTML in simple Hinglish, using the analogy of a government office where every room has a nameplate versus one where every room is just numbered. Explain how this helps both blind users and Google. Keep it under 150 words.

SESSION

05

60 MIN · CSS FUNDAMENTALS

Introduction to CSS3

◆ Learning Objectives

- Explain the three ways to add CSS to an HTML page and when to use each
- Write CSS rules using element, class, ID, descendant, and pseudo-class selectors
- Explain CSS specificity and predict which rule wins when two rules conflict
- Apply colour values using named colours, hex codes, RGB, and RGBA
- Set font-family, font-size, font-weight, line-height, and text-align properties
- Use CSS custom properties (variables) to centralise theme colours
- Build a working light and dark mode colour theme switch

01 Three Ways to Add CSS

CSS can reach an HTML page in three ways. Every professional developer uses only one of them as their primary method — external stylesheets — and knowing why the other two exist (and when they are useful) is your first step toward writing maintainable front-end code.

<!-- METHOD 1: Inline CSS — applied to a single element only -->

<h1 style='color: #E34C26; font-size: 32px;'>Inline Style</h1>

Highest specificity; hard to maintain; avoid for layout

<!-- METHOD 2: Internal CSS — inside a <style> tag in <head> -->

<style>

h1 { color: #E34C26; }

</style>

Styles apply to this one HTML file only

Element selector targeting all h1 tags on the page

Good for single-page experiments and email templates

<!-- METHOD 3: External CSS — linked file (industry standard) -->

<link rel='stylesheet' href='style.css'>

One CSS file can style every page of the entire site

Inline or Internal CSS

Fast to write for quick tests. Tightly coupled to one page — if you want to change the brand colour, you edit 50 different files. Mixes content and presentation in the same document. Use for: rapid prototyping, email HTML, or a single-file demo.

External CSS (href link)

One .css file styles every page of the site — update one line and the whole site changes. Browsers cache the file, so repeat visitors load pages faster. Complete separation of HTML structure and CSS style — the industry standard for every multi-page website.

02 Selectors — Targeting Elements

A CSS rule has two parts: a selector that targets one or more HTML elements, and a declaration block (inside curly braces) that lists the properties to change. The selector is where most of CSS's power lives — learning to write precise selectors means you can style any element on any page without touching its HTML.

```
p { color: #333333; }
.card { background-color: #f5f5f5; }
#header { background-color: #000000; }
```

Element selector — targets every p tag on the page

Class selector — targets elements with class='card'

ID selector — targets the single element with id='header'

```
nav a { text-decoration: none; }
h1, h2, h3 { font-family: Arial, sans-serif; }
```

Descendant selector — all a tags inside a nav element

Multiple selector — same rule applied to all three tags

```
a:hover { color: #E34C26; }
input:focus { border: 2px solid #0077cc; }
```

Pseudo-class — applies only when the link is hovered

Pseudo-class — applies when an input is active/focused

Selector Type	Syntax	Specificity Score
Element	p { }	1
Class	.card { }	10
ID	#hero { }	100
Inline	style='...'	1000
Descendant	nav a { }	11 (1+10 if a has a class)
Multiple	h1, h2 { }	1 each — not combined

CONCEPT — CSS Specificity

When two CSS rules target the same element and conflict, specificity determines which one wins. Think of it like a priority queue at a counter: VIP pass (inline style, 1000 points) beats Fast Track ticket (ID selector, 100 points) beats Regular ticket (class selector, 10 points) beats General entry (element selector, 1 point). If two rules tie on specificity, the one written last in the file wins — this is the Cascade part of CSS.

03 Colours & Typography

CSS offers four ways to write colour values. Hex codes are most common in design handoffs and developer tools. RGB and RGBA are preferred when you need transparency. Named colours are readable in code but limited to 147 predefined options — fine for quick tests, rarely used in production.

<code>color: red;</code>	Named colour — 147 options in CSS (coral, teal, crimson...)
<code>color: #E34C26;</code>	Hex — 6-digit code; most common in Figma/Zeplin handoffs
<code>color: rgb(227, 76, 38);</code>	RGB — same colour as above in decimal
<code>color: rgba(227, 76, 38, 0.5);</code>	RGBA — 4th value is opacity (0 = transparent, 1 = opaque)
<code>background-color: #f0f4f8;</code>	Sets element background
<code>border: 2px solid #cccccc;</code>	Border with colour — shorthand for width, style, and color

◆ Typography Properties

<code>font-family: 'Segoe UI', Arial, sans-serif;</code>	Font stack — browser tries each in order; sans-serif is the fallback
<code>font-size: 16px;</code>	Fixed pixel size
<code>font-size: 1rem;</code>	Relative to root font size — better for accessibility and scaling
<code>font-weight: 700;</code>	400 = normal, 600 = semi-bold, 700 = bold, 900 = black
<code>line-height: 1.6;</code>	1.6x the font size — ideal for body text readability
<code>text-align: center;</code>	left, right, center, or justify
<code>text-transform: uppercase;</code>	ALL CAPS without changing the HTML — also lowercase, capitalize
<code>letter-spacing: 0.05em;</code>	Adds space between characters — em is relative to font-size

PRO TIP — CSS Custom Properties (Variables)

Define your brand colours once in `:root` and reuse them everywhere. When the brand changes its primary colour, you update one line — not 200. This is how design systems work at companies like Razorpay and Swiggy.

<code>:root {</code>	Declare variables on the root element — available everywhere
<code>--primary: #E34C26;</code>	Custom property — name must start with --
<code>--bg: #ffffff;</code>	Background colour variable
<code>--text: #111111;</code>	Text colour variable
<code>}</code>	
<code>h1 { color: var(--primary); }</code>	Use a variable with <code>var()</code>
<code>body { background-color: var(--bg); }</code>	Consistent background across all elements

04 Demo — Light / Dark Mode Theme Switch

The classic CSS colour demo: two sets of colour values controlled by a class on the body tag, toggled by a single button click. This pattern — CSS variables + a class toggle — is exactly how major apps like Twitter, Notion, and Zerodha implement their dark mode in production.

```
/* style.css */
body {
  background-color: #ffffff;
  color: #111111;
  transition: background-color 0.3s ease, color 0.3s ease;
}
body.dark {
  background-color: #1a1a2e;
  color: #e0e0e0;
}
/* index.html – inside <body> */
<button id='toggle'>Switch Theme</button>
/* app.js – paste before </body> in a <script> tag */
document.getElementById('toggle').addEventListener('click', () => {
  document.body.classList.toggle('dark');
});
```

Default state (light mode)
 White background
 Dark text
 Smooth animation when switching

Dark mode — applied when body has class='dark'
 Dark navy background
 Light grey text

The toggle button

Listen for click

Add 'dark' if missing, remove if present

TRY THIS LIVE — Build the Theme Switch

Create two files: index.html and style.css. In the HTML, link the CSS file, add a button with id='toggle', add a few h1, p, and a tags so you can see the colour change. Add the dark mode CSS rules. Then paste the three-line JavaScript in a script tag before closing body. Save and test — click the button and watch the page transition. Then open DevTools and watch the 'dark' class appear and disappear on the body element.

CONVERSION LIFT FROM 1-SECOND CSS SPEED IMPROVEMENT

27%

Source: Google/Deloitte Mobile Speed Study, 2020 — render-blocking CSS is one of the top causes of slow pages.

WATCH — VIDEO REFERENCE (HINDI / ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Internet Technologies — lecture on CSS and Web Styling (IIT Kharagpur). Trainer: verify lecture

number at nptel.ac.in before class.

Link — <https://nptel.ac.in/courses/106105084>

WHERE TO GO NEXT

You can now write CSS rules, target any element precisely, and apply colours and typography — the visual layer of every web page.

Explore deeper — Bookmark the MDN CSS reference at developer.mozilla.org/en-US/docs/Web/CSS and the CSS-Tricks almanac at css-tricks.com/almanac — two tabs every front-end developer keeps open.

Try on your own — Pick any TOPS Technologies course card on tops-int.com. Recreate its font choices, colour values, and text sizes using only the CSS properties from this session. Use DevTools to inspect the exact values.

Connect the dots — Session 6 introduces the Box Model — margin, padding, border, and display — which controls how elements are sized and spaced around the content you styled today.

Going further — Look up CSS custom properties (variables) and the `prefers-color-scheme` media query — together they let the page automatically match the user's system-level dark mode preference without any JavaScript.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a CSS teacher. Explain CSS specificity in simple Hinglish using the analogy of priority passes at an event — VIP, Fast Track, Regular, and General. Give a real example of two conflicting CSS rules and tell me which one wins and why. Keep it under 150 words.

SESSION

06

60 MIN · CSS LAYOUT

Box Model & Positioning

◆ Learning Objectives

- Explain the CSS box model and name its four layers from inside to outside
- Apply margin, padding, border, and border-radius to control spacing and shape
- Set box-sizing: border-box and explain why every CSS file starts with it
- Use the display property to control how elements participate in page flow
- Position elements using relative, absolute, fixed, and sticky positioning
- Combine box model and positioning to build a styled profile card with a badge

01 The Box Model — Every Element Is a Box

In CSS, every element on the page — a heading, an image, a button — is rendered as a rectangular box. That box has four layers stacked from the inside out: content (the actual text or image), padding (space between the content and the border), border (the visible edge), and margin (space between this element and its neighbours). Understanding this model is the key to controlling layout without guesswork.

CONCEPT — The Four Layers

Think of a visiting card. The printed text — your name and number — is the content. The blank space printed around the text inside the card boundary is padding. The physical card edge is the border. The gap you leave between this card and other items on your desk is the margin. Change any of these and the element's footprint on the page changes.

```
/* Margin — space outside the border */
.card { margin: 20px; }           All four sides equal
.card { margin: 16px 24px; }      16px top/bottom, 24px left/right
.card { margin: 0 auto; }         Centres a block element (requires a set width)

/* Padding — space inside the border */
.card { padding: 24px; }          All four sides equal
.card { padding: 16px 24px; }     16px top/bottom, 24px left/right

/* Border */
.card { border: 1px solid #e0e0e0; } Shorthand: width, style, colour
.card { border-radius: 12px; }     Rounds all four corners
.card { box-shadow: 0 4px 16px rgba(0,0,0,0.08); }
    x-offset y-offset blur colour — adds depth
```

PRO TIP — Always Start with box-sizing: border-box

By default, CSS adds padding and border on top of your set width. So width: 300px + padding: 24px = a 348px-wide box — not what you intended. With box-sizing: border-box, padding and border are included inside the 300px. Add this to the very first line of every CSS file you write: `* { box-sizing: border-box; margin: 0; padding: 0; }`

02 The display Property

The display property controls whether an element starts on a new line, how much width it takes, and whether you can set width and height on it. Every HTML tag has a default display value — div is block, span is inline — but you can override any of them with CSS. This is how navigation links (which are inline anchor tags) become horizontal menu items.

<code>div { display: block; }</code>	Full parent width, starts on new line, respects width/height
<code>span { display: inline; }</code>	Flows inside text — width and height have no effect
<code>img { display: inline-block; }</code>	Flows with text but respects width and height
<code>.menu { display: none; }</code>	Removes element from layout — invisible, takes no space
<code>.nav { display: flex; }</code>	Enables Flexbox — children become flex items (Session 7)
<code>.page { display: grid; }</code>	Enables CSS Grid — children become grid items (Session 7)

Value	Behaviour	Default for
block	Full width, new line, respects width/height	div, p, h1–h6, header, section
inline	Text flow, no width/height control	span, a, strong, em
inline-block	Text flow + respects width and height	img, button, input
none	Removed from layout entirely — invisible, no space	No HTML default; used to hide elements
flex	Flexbox container — children become flex items	No HTML default
grid	Grid container — children become grid items	No HTML default

03 Positioning — Taking Control of the Flow

By default, every element sits in normal document flow — block elements stack vertically, inline elements flow like text. The position property lets you break elements out of that flow and place them precisely anywhere on the page. There are four values beyond the default, each with a specific use case.

```
/* relative — offset from normal position, space is still reserved */
.icon { position: relative; top: 2px; }           Nudges icon down 2px without affecting surrounding
                                                    elements

/* absolute — anchored to nearest non-static ancestor */
.badge { position: absolute; top: 12px; right: 12px; }
    Stuck to the top-right of its positioned parent

/* fixed — anchored to the viewport, stays on scroll */
.navbar { position: fixed; top: 0; left: 0; width: 100%; }
    Sticky top navigation bar — stays visible while user scrolls

/* sticky — normal flow until scroll threshold */
.section-title { position: sticky; top: 0; background: #fff; }
    Sticks to top while its parent section is in the viewport

/* z-index — stacking order; higher = closer to user */
.modal { position: fixed; z-index: 1000; }       Modal appears above all other content
.overlay { position: fixed; z-index: 999; }      Dark backdrop sits just behind the modal
```

CONCEPT — How absolute Positioning Works

An element with position: absolute is removed from normal flow and anchored to the nearest ancestor that has any position value other than static. If no such ancestor exists, it anchors to the browser window. This is why the parent of a badge or tooltip almost always needs position: relative — it creates the positioning boundary without visually moving anything.

TRY THIS LIVE — Add a Badge to a Card

Create a div.card with some text inside. Add position: relative to the card. Inside the card, add a span.badge with the text 'New'. Give the badge position: absolute; top: 12px; right: 12px. Style it with a background colour and border-radius: 20px. The badge should appear in the top-right corner of the card, anchored precisely regardless of the card's content.

04 Demo — Profile Card Layout

The profile card is the definitive box model exercise — it combines margin (page centering), padding (inner spacing), border-radius (card shape), box-shadow (depth), and absolute positioning (badge placement) into one practical component. This same pattern appears in Naukri.com recruiter cards, LinkedIn contact cards, and every HR platform built in India.

```
* { box-sizing: border-box; margin: 0; padding: 0; }
```

Global reset — put this at the top of every CSS file

<code>.profile-card {</code>	Card container
<code>width: 300px;</code>	Fixed width
<code>background: #ffffff;</code>	White background
<code>border: 1px solid #e0e0e0;</code>	Subtle border
<code>border-radius: 16px;</code>	Rounded corners
<code>padding: 32px 24px;</code>	Breathing room inside the card
<code>margin: 48px auto;</code>	Centred with top/bottom spacing
<code>position: relative;</code>	Positioning context — required for the badge
<code>box-shadow: 0 4px 20px rgba(0,0,0,0.08);</code>	Soft shadow for depth
<code>text-align: center; }</code>	Centre all text in the card

<code>.profile-card img {</code>	Profile photo
<code>width: 88px; height: 88px;</code>	Square before border-radius is applied
<code>border-radius: 50%;</code>	Makes the square a perfect circle
<code>display: block; margin: 0 auto 16px; }</code>	Centred with gap below
<code>.profile-card h2 { font-size: 20px; margin-bottom: 4px; }</code>	
Name — small gap before the role line	
<code>.profile-card p { color: #777777; margin-bottom: 20px; }</code>	
Role/bio — muted colour	
<code>.badge {</code>	Floating label — positioned absolutely
<code>position: absolute; top: 16px; right: 16px;</code>	Anchored to profile-card (its positioned ancestor)
<code>background: #E34C26; color: #ffffff;</code>	Brand colour
<code>padding: 4px 12px; border-radius: 20px; font-size: 11px; }</code>	
Pill shape	

TRY THIS LIVE — Build the Full Profile Card

Create index.html and style.css. In the HTML, add: a div.profile-card containing a span.badge ('Available'), an img (use any placeholder from picsum.photos), an h2 with your name, a p with your role, and an anchor button. Paste both CSS code blocks above. Open in Chrome — you should see a centred white card with a rounded profile photo and an orange badge in the top-right corner.

“Once you see every element as a box, CSS layout stops being magic and starts being math.”

[WATCH — VIDEO REFERENCE \(HINDI / ENGLISH\)](#)

Channel — NPTEL (official — IITs / IISc)

Watch — Internet Technologies — lecture on CSS Box Model and Layout (IIT Kharagpur). Trainer: verify lecture number at nptel.ac.in before class.

Link — <https://nptel.ac.in/courses/106105084>

WHERE TO GO NEXT

You can now size, space, and position any element on a page precisely — the foundation of every real-world CSS layout.

Explore deeper — Open the MDN Box Model guide at developer.mozilla.org/en-US/docs/Learn/CSS/Building_blocks/The_box_model — the interactive diagram is the clearest visual explanation available.

Try on your own — Open any TOPS course listing card on tops-int.com in DevTools. Click on it and inspect the Box Model diagram at the bottom of the Computed panel — read its exact margin, padding, and border values.

Connect the dots — Session 7 introduces Flexbox and Grid — two display values that turn a simple block of elements into powerful, responsive layouts using only CSS.

Going further — Look up CSS logical properties (`margin-inline`, `padding-block`) — they are the modern, direction-aware replacement for `margin-left`, `margin-right`, etc., and are essential for building multilingual pages that support right-to-left text.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a CSS teacher. Explain the CSS box model in simple Hinglish using the analogy of a courier parcel — the item inside, the bubble wrap, the box itself, and the space the delivery person leaves between parcels. Then explain what box-sizing: border-box changes and why developers prefer it. Keep it under 150 words.

SESSION

07

60 MIN · CSS LAYOUT

Flexbox & Grid

◆ Learning Objectives

- Enable Flexbox on a container and explain the main axis and cross axis
- Control child spacing and alignment with justify-content, align-items, and gap
- Let flex children grow, shrink, and wrap using flex, flex-grow, and flex-wrap
- Build a responsive social icon gallery using Flexbox
- Create a CSS Grid layout using grid-template-columns and the fr unit
- Span grid items across multiple columns and rows
- Choose correctly between Flexbox and Grid for a given layout

01 Why Flexbox Exists

Before Flexbox, centring a div vertically inside its parent required multi-step hacks involving absolute positioning and negative margins. Building a nav bar with items evenly spaced required float: left and clearfix workarounds. Flexbox — introduced in CSS3 — solved both in one line each. It is now the standard tool for any one-dimensional layout: a row of buttons, a navigation bar, an icon gallery, a card strip.

CONCEPT — Main Axis vs Cross Axis

Flexbox works on two perpendicular axes. The main axis is the direction flex items are arranged — row (left to right) by default, or column (top to bottom). The cross axis is perpendicular to it. justify-content controls spacing along the main axis. align-items controls alignment along the cross axis. Everything in Flexbox flows from understanding which axis you are working on.

Flexbox — One Dimension

Works in one direction at a time: either a row or a column. Content-first — items size themselves based on their content, then fill available space. Best for: navigation bars, icon galleries, button groups, card strips, form rows.

CSS Grid — Two Dimensions

Controls rows and columns simultaneously. Layout-first — you define the grid structure; content fills the cells. Best for: full page layouts, photo grids, dashboard panels, magazine-style designs with items of varying size.

02 Flexbox — Container Properties

Flexbox is enabled on the parent (the container), not on the children. Adding display: flex to a container instantly makes all its direct children flex items. Everything else — direction, spacing, alignment, wrapping — is set on the container too. The children only need item-level properties when you want to override the container's defaults for a specific child.

<code>.container { display: flex; }</code>	Enables Flexbox — all direct children are now flex items
<code>.container { flex-direction: row; }</code>	Main axis: left-to-right (default)
<code>.container { flex-direction: column; }</code>	Main axis: top-to-bottom (stacks items vertically)
<code>.container { justify-content: flex-start; }</code>	Items packed to the start of the main axis (default)
<code>.container { justify-content: center; }</code>	Items centred along the main axis
<code>.container { justify-content: space-between; }</code>	First and last at edges; equal space between all others
<code>.container { justify-content: space-evenly; }</code>	Equal space before, between, and after every item
<code>.container { align-items: center; }</code>	Items centred on the cross axis (vertical if row)
<code>.container { align-items: stretch; }</code>	Items stretch to fill container height (default)
<code>.container { flex-wrap: wrap; }</code>	Items wrap to a new line when they run out of space
<code>.container { gap: 16px; }</code>	Equal space between all flex items — replaces margins on children
<code>.container { gap: 12px 20px; }</code>	Row gap 12px, column gap 20px

Property	Values	What It Controls
flex-direction	row, row-reverse, column, column-reverse	Which way items are arranged (the main axis)
justify-content	flex-start, flex-end, center, space-between, space-evenly	Spacing and alignment along the main axis
align-items	flex-start, flex-end, center, stretch, baseline	Alignment on the cross axis
flex-wrap	nowrap (default), wrap, wrap-reverse	Whether items can wrap to a new row/column
gap	length (px, rem, %)	Space between items — cleaner than margin on children

03 Flexbox Item Properties & Demo

While containers control the overall layout, individual flex items can override those rules for themselves. The flex shorthand is the most important item property — it tells the browser how an item should grow to fill available space, shrink if space is tight, and what its starting size should be before either of those happen.

<code>.item { flex: 1; }</code>	Shorthand for flex-grow: 1, flex-shrink: 1, flex-basis: 0 — item takes equal share of space
<code>.item { flex: 2; }</code>	Takes twice as much available space as a flex: 1 sibling
<code>.item { flex: 0 0 200px; }</code>	Fixed 200px — will not grow or shrink

<code>.item { flex-grow: 1; }</code>	Grows to fill spare space; 0 = does not grow (default)
<code>.item { flex-shrink: 0; }</code>	Never shrinks below its set width; 1 = can shrink (default)
<code>.item { flex-basis: 160px; }</code>	Initial size before growing or shrinking
<code>.item { align-self: flex-end; }</code>	Overrides align-items for this one item only
<code>.item { order: -1; }</code>	Moves this item before all order: 0 siblings (visual only)

<code>/* Social icon gallery – flex container */</code>	
<code>.icon-gallery {</code>	Gallery wrapper
<code>display: flex;</code>	Enable Flexbox
<code>flex-wrap: wrap;</code>	Icons wrap to next line when space runs out
<code>justify-content: center;</code>	Centre the icons horizontally
<code>gap: 20px;</code>	Equal spacing between all icons
<code>padding: 24px; }</code>	Inner spacing around the gallery
<code>.icon-item {</code>	Each icon + label unit
<code>display: flex;</code>	Stack image and label vertically
<code>flex-direction: column;</code>	Main axis: top to bottom
<code>align-items: center;</code>	Centre image and label on cross axis
<code>gap: 8px;</code>	Space between icon and label
<code>width: 80px; }</code>	Fixed width so all items are uniform

TRY THIS LIVE — Responsive Social Icon Gallery

Create a `div.icon-gallery`. Inside, add 6 `div.icon-item` elements — each with an `img` (use emoji or a 48×48 placeholder image) and a `p` label (Instagram, LinkedIn, GitHub, etc.). Apply the gallery CSS above. Resize the browser window and watch the icons wrap to a second row automatically. This is the same pattern used for app icon grids on mobile home screens.

04 CSS Grid — Two-Dimensional Layouts

CSS Grid is the first CSS layout system designed for two dimensions simultaneously. You define the columns and rows upfront, and the browser places items into the resulting cells. The killer feature is the `fr` unit — fractional unit — which divides available space proportionally without you ever doing pixel math. Most modern page layouts (course grids, dashboards, article feeds) are built with Grid.

<code>.grid { display: grid; }</code>	Enable CSS Grid
<code>.grid { grid-template-columns: 1fr 1fr 1fr; }</code>	3 equal columns — <code>fr</code> = fractional unit
<code>.grid { grid-template-columns: repeat(3, 1fr); }</code>	Identical to above — shorter with <code>repeat()</code>
<code>.grid { grid-template-columns: 240px 1fr 1fr; }</code>	Fixed sidebar + 2 flexible columns

```
.grid { grid-template-columns: repeat(auto-fill, minmax(200px, 1fr)); }
```

Auto-responsive: as many 200px+ columns as fit

```
.grid { gap: 20px; }
```

Space between all rows and columns

```
/* Spanning items across cells */
```

```
.featured { grid-column: 1 / 3; }
```

Spans from column line 1 to column line 3 (2 columns wide)

```
.banner { grid-column: 1 / -1; }
```

Spans all columns (-1 = last column line)

```
.tall { grid-row: 1 / 3; }
```

Spans 2 rows tall

CONCEPT — The fr Unit

fr stands for fractional unit. After fixed-size columns are placed, the remaining space is divided into fractions. grid-template-columns: 1fr 2fr gives two columns where the second is exactly twice the width of the first — always, at any viewport size. This completely replaces percentage-based column math and eliminates the need to calculate gaps manually.

PRO TIP — Grid vs Flexbox Decision Rule

If your layout has one direction (a row of buttons, a vertical menu, a list of cards in a single row) — use Flexbox. If your layout has both rows and columns (a photo gallery, a page with a header, sidebar, main content, and footer, a dashboard) — use Grid. In practice, most pages use both: Grid for the page structure and Flexbox for the components inside each grid area.

TRY THIS LIVE — 3-Column Course Card Grid

Create a div.course-grid. Inside, add 6 div.course-card elements, each with an h3 (course name) and a p (duration). Apply: display: grid; grid-template-columns: repeat(3, 1fr); gap: 24px to the grid, and some padding and background to the cards. Make the first card span two columns with grid-column: 1 / 3. This is the layout pattern for tops-int.com's course listing page.

CSS GRID ADOPTION ON TOP WEBSITES

80%+

Source: HTTP Archive Web Almanac, 2024 — up from less than 2% of websites in 2017.

WATCH — VIDEO REFERENCE (HINDI / ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Internet Technologies — lecture on Advanced CSS Layout (IIT Kharagpur). Trainer: verify lecture number at nptel.ac.in before class.

Link — <https://nptel.ac.in/courses/106105084>

WHERE TO GO NEXT

You can now build any one-dimensional or two-dimensional layout cleanly — the skill that separates developers who fight CSS from those who command it.

Explore deeper — Play the free browser game Flexbox Froggy (flexboxfroggy.com) and CSS Grid Garden (cssgridgarden.com) — both teach layout through puzzles and are used by front-end teams at major companies for onboarding.

Try on your own — Recreate the course card grid on tops-int.com using CSS Grid. Count the columns, estimate the gap, and match the layout as closely as possible using only the properties from this session.

Connect the dots — Session 8 adds media queries — you will make your Flexbox and Grid layouts respond to screen size, turning desktop grids into single-column mobile stacks with a few extra lines of CSS.

Going further — Look up CSS Subgrid (`grid-template-columns: subgrid`) — it allows nested grids to align to the parent grid's tracks and is now supported in all major browsers since 2023.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a CSS layout teacher. Explain the difference between CSS Flexbox and CSS Grid in simple Hinglish using a real-world analogy from India — for example, compare a train compartment (one direction, Flexbox) to a cricket field seating plan (rows and columns, Grid). Give one example of a UI component best suited to each. Keep it under 150 words.

SESSION

08

60 MIN · RESPONSIVE CSS

Responsive Design

◆ Learning Objectives

- Explain what responsive design is and why it is especially critical for Indian web users
- Add the viewport meta tag and explain what happens without it
- Write CSS media queries using min-width breakpoints
- Apply a mobile-first design approach and explain why it is preferred
- Convert the Session 6 profile card into a fully responsive component
- Use Chrome DevTools device simulation to test across multiple screen sizes

01 Why Responsive Design Is Non-Negotiable in India

In India, the web is mobile. Most Indians experienced the internet for the first time on a smartphone — not a laptop — and the majority have never owned a desktop computer. When you build a webpage that only looks good at 1280px wide, you are building for the minority. Responsive design means the same HTML and CSS works correctly on a 360px phone screen, a 768px tablet, and a 1440px desktop monitor — no separate mobile site required.

INDIA'S INTERNET TRAFFIC FROM MOBILE

95%

Source: StatCounter GlobalStats India, 2024 — making mobile-first layout a business requirement, not a design preference.

CONCEPT — Three Pillars of Responsive Design

Responsive design rests on three techniques working together. First: a fluid layout — widths in percentages or fr units that stretch with the viewport. Second: flexible media — images and videos that scale down on small screens (max-width: 100%). Third: media queries — CSS rules that apply only when the screen reaches a certain width. This session covers all three.

02 The Viewport Meta Tag

Mobile browsers have a quirk: by default, they render pages as if the screen were 980px wide, then scale the result down to fit the physical screen. This makes a desktop-width page legible but tiny — users must pinch and zoom to read anything. The viewport meta tag tells the browser to use the actual device width instead, and to not scale the page down.

```

<!-- Put this inside your <head> tag – it must be present on every page -->
<meta name='viewport' content='width=device-width, initial-scale=1.0'>
    width=device-width: use actual screen width; initial-scale=1: no default zoom
<!-- Without it – what mobile browsers do instead -->
<!-- Renders at 980px wide then scales down – text is tiny, no zoom benefit -->
<!-- With it – every CSS pixel = 1 device pixel at 100% zoom -->

```

WARNING — Missing Viewport Tag Breaks Every Media Query

If the viewport meta tag is absent, media queries do not fire at the right widths — your @media (max-width: 768px) rule triggers at 980px (the fake desktop width) instead of the actual device width. Every responsive layout you write is broken without this one meta tag. The VS Code Emmet boilerplate (! + Tab) includes it automatically — do not delete it.

03 Media Queries — CSS That Reacts to Screen Size

A media query is a conditional CSS block that applies only when the screen meets a condition — typically a minimum or maximum width. Everything inside the @media block overrides the base styles for that screen size. Using min-width (mobile-first approach) is the industry standard: you write the simplest layout first for the smallest screen, then progressively enhance it for larger screens.

```

/* Base styles – written for mobile first (default) */
.grid { grid-template-columns: 1fr; }           One column on mobile
.nav ul { display: none; }                     Hide nav links on small screens
h1 { font-size: 1.5rem; }                     Smaller heading on mobile

/* Tablet – 768px and wider */
@media (min-width: 768px) {                     This block only applies at 768px+
    .grid { grid-template-columns: repeat(2, 1fr); }
        Two columns on tablet
    h1 { font-size: 2rem; } }                 Larger heading at tablet width

/* Laptop – 1024px and wider */
@media (min-width: 1024px) {
    .grid { grid-template-columns: repeat(3, 1fr); }
        Three columns on laptop
    .nav ul { display: flex; } }             Show full nav on desktop

/* Desktop – 1280px and wider */
@media (min-width: 1280px) {
    .container { max-width: 1200px; margin: 0 auto; } }
        Cap width and centre on large screens

```

Breakpoint	min-width	Target Devices
Mobile (base)	None — default	All phones: 320px -> 767px
Tablet	768px	iPad, large phones in landscape
Laptop	1024px	Small laptops, tablets landscape
Desktop	1280px	Standard desktops and laptops
Widescreen	1536px	Large monitors (optional — use max-width on container instead)

CONCEPT — Mobile-First vs Desktop-First

Mobile-first means you write base styles for the smallest screen, then use min-width media queries to add complexity as the screen grows. Desktop-first means the opposite — you start wide and use max-width queries to simplify for smaller screens. Mobile-first is the industry standard today for three reasons: it forces you to prioritise content, media queries only add rules (no overriding), and browsers load fewer CSS rules on the slowest devices.

04 Demo — Make the Profile Card Responsive

Take the profile card built in Session 6 and make it fully responsive. On mobile it should be full-width with a smaller photo and tighter padding. On tablet and up, it returns to the fixed 300px centred card. This same pattern — full width on mobile, fixed width centred on desktop — is how every card in every Indian app from Swiggy to Zerodha behaves.

```
/* Mobile-first base — the card fills the screen */
.profile-card {
  width: 100%;                Full width on mobile
  max-width: 400px;          Never wider than 400px even on a large phone
  padding: 24px 16px;        Tighter padding on small screens
  margin: 24px auto;         Centred with smaller vertical gap
  .profile-card img { width: 72px; height: 72px; }
    Smaller photo on mobile
  .profile-card h2 { font-size: 1.1rem; }    Slightly smaller name on small screen

/* Tablet and up — restore the designed card */
@media (min-width: 768px) {
  .profile-card {
    width: 300px;              Fixed 300px width returns
    padding: 32px 24px;        Restore original padding
    margin: 48px auto;         More vertical breathing room
  }
}
```

```
.profile-card img { width: 88px; height: 88px; }
```

Full-size photo restored

```
.profile-card h2 { font-size: 1.25rem; } }
```

Full heading size

TRY THIS LIVE — Responsive Profile Card

Add the media query block above to your Session 6 profile card CSS. Open in Chrome -> DevTools -> press Ctrl+Shift+M to enter Device Simulation mode. Select 'iPhone SE' from the device dropdown — the card should now fill the screen width with a smaller photo. Switch to 'iPad Air' — the fixed 300px card should reappear. Try dragging the resize handle to see the layout switch at exactly 768px.

PRO TIP — Chrome DevTools Device Simulation

In DevTools, press Ctrl+Shift+M (or click the phone icon) to enter Device Simulation mode. Select any device from the dropdown — iPhone 14, Galaxy S21, iPad Air — or type a custom width. You can also throttle network speed to 'Slow 3G' to test how your page loads on the connections most Indian users are on. This is how every front-end developer QA-tests responsive design without owning 10 devices.

CASE STUDY — Flipkart Lite (India)

Context — Flipkart, India's largest e-commerce platform at the time, had over 100 million registered users, with more than 75% of traffic arriving on mobile — often on 2G or 3G connections in Tier-2 and Tier-3 cities.

Challenge — The full desktop site timed out on slow connections. The native Android app required downloading over 10MB before users could browse. High bounce rates were costing Flipkart sales in price-sensitive markets.

What they did — Flipkart built Flipkart Lite — a mobile-first responsive Progressive Web App with a load target under 3 seconds on 2G. They used responsive images, minimal blocking CSS, and mobile-first media queries to serve the simplest layout first and progressively enhance for faster connections.

Result — Users who added Flipkart Lite to their home screen converted at significantly higher rates than browser users. Data usage dropped 3x compared to the native Android app, making the experience viable on limited data plans.

Takeaway — Mobile-first responsive design is not a technical preference in India — it is the difference between having customers and not having them.

Source: Google I/O 2016 Developer Showcase — Flipkart Lite (developers.google.com)

WATCH — VIDEO REFERENCE (HINDI / ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Internet Technologies — lecture on Responsive Web Design and Mobile-First Development (IIT Kharagpur). Trainer: verify lecture number at nptel.ac.in before class.

Link — <https://nptel.ac.in/courses/106105084>

WHERE TO GO NEXT

You can now build layouts that work correctly on every screen — the baseline expectation for any web developer entering the industry in 2025.

Explore deeper — Read Google's Web.dev guide 'Responsive Design' at web.dev/learn/design — it is the most current, practical reference and is what Google uses internally to train its own developers.

Try on your own — Open tops-int.com on your phone and inspect the mobile layout in DevTools. Find three elements that change between mobile and desktop — guess which CSS property changes and at which breakpoint, then verify using the Computed panel.

Connect the dots — Session 9 introduces Bootstrap — a CSS framework that gives you pre-built responsive grids and components so you do not have to write every media query from scratch in production.

Going further — Look up the CSS Container Query (@container) — a newer alternative to media queries that makes components respond to the size of their own parent container rather than the viewport, enabling truly portable responsive components.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a web development teacher. Explain mobile-first responsive design in simple Hinglish using the example of how Jio changed how Indians use the internet. Explain what media queries do, why we write for mobile first, and what happens when we skip the viewport meta tag. Keep it under 150 words.

SESSION

09

60 MIN · CSS FRAMEWORK

Bootstrap Basics

◆ Learning Objectives

- Explain what Bootstrap is, what problem it solves, and how to add it via CDN
- Use .container and .container-fluid to control page width correctly
- Understand Bootstrap's 12-column grid model and calculate column spans
- Apply responsive column classes (col-md-6, col-lg-4) for breakpoint-specific layouts
- Build a 2-column profile layout with image and link list using Bootstrap Grid

01 What Is Bootstrap and Why Use It?

Bootstrap is a free, open-source CSS framework that gives you a pre-built responsive grid system, ready-made components (navbars, cards, buttons, forms), and hundreds of utility classes — all in one file you link from a CDN. Without Bootstrap, every developer writes their own media queries, their own card styles, and their own button variants from scratch. Bootstrap standardises all of that so a team can build consistent UIs faster. It is the most widely used CSS framework in the world.

CONCEPT — What a CSS Framework Gives You

A CSS framework is a pre-written stylesheet you load into your project. It does not replace your custom CSS — it sits below it. Bootstrap's stylesheet gives you: (1) a normalised baseline so all browsers render the same, (2) a 12-column responsive grid, (3) fully styled, accessible components, and (4) utility classes for spacing, colour, and typography. You extend it; you do not rewrite it.

```
<!-- Paste inside <head> – Bootstrap 5 CSS via CDN -->
<link rel='stylesheet'
href='https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css'>
    All Bootstrap styles — load before your custom CSS
<!-- Paste just before </body> – Bootstrap JS (needed for interactive components) -->
<script
src='https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js'></s
cript>
    Includes Popper.js — needed for dropdowns, tooltips, navbar toggle
<!-- Check getbootstrap.com for the latest version number -->
```

WEBSITES USING BOOTSTRAP

19.2%

Source: W3Techs, 2024 — Bootstrap is the most widely used CSS framework on the internet.

02 Containers

Every Bootstrap layout starts with a container. The container class creates a responsive, fixed-width wrapper that has automatic horizontal margins and predefined max-widths at each breakpoint. Container-fluid always spans 100% of the viewport width. All rows and columns must be direct children of a container — wrapping them correctly is the most common Bootstrap mistake beginners make.

<code><div class='container'></code>	Responsive fixed width — max-width changes at each breakpoint (540px, 720px, 960px, 1140px, 1320px)
<code><!-- rows go here --></code>	
<code></div></code>	
<code><div class='container-fluid'></code>	Always 100% viewport width — used for full-bleed sections (hero banners, navbars)
<code><!-- rows go here --></code>	
<code></div></code>	
<code><div class='container py-5'></code>	Containers accept utility classes — py-5 adds padding top and bottom (covered in Session 11)
<code></div></code>	

03 The 12-Column Grid System

Bootstrap divides every container into 12 invisible columns. You assign each element a number of those columns using col-{n} classes. The numbers across one row must add up to 12 — or less if you want gaps. Breakpoint prefixes (col-md-6, col-lg-4) let you change how many columns an element spans at different screen widths, giving you full responsive control without writing a single media query.

<code><!-- Basic structure --></code>	
<code><div class='container'></code>	Step 1: container
<code><div class='row'></code>	Step 2: row — creates a Flexbox row with negative margins
<code><div class='col-6'>Left half</div></code>	Step 3: col — 6 of 12 columns = 50%
<code><div class='col-6'>Right half</div></code>	6 + 6 = 12 — fills the full row
<code></div></code>	
<code><!-- Responsive columns --></code>	
<code><div class='col-12 col-md-6'></code>	Full width on mobile; half width on md (768px) and up
<code><div class='col-12 col-md-6 col-lg-4'></code>	Full -> half -> third as screen grows
<code><div class='col-12 col-sm-6 col-lg-3'></code>	Full -> half at sm -> quarter at lg

col class	Columns Spanned	Width
col-12	12 of 12	100% — full row
col-6	6 of 12	50% — two equal columns
col-4	4 of 12	33.33% — three equal columns
col-3	3 of 12	25% — four equal columns
col-8 + col-4	8 + 4 = 12	66% + 33% — sidebar layout
col (no number)	auto	Equal share of remaining space

Prefix	Breakpoint Name	min-width
(none)	xs — extra small (mobile default)	< 576px
sm	Small	576px
md	Medium	768px
lg	Large	992px
xl	Extra large	1200px
xxl	Extra extra large	1400px

PRO TIP — Always Add row-cols or g-{n} for Gutters

Bootstrap columns have horizontal padding (called gutters) by default. Adding g-3 or g-4 to the row class sets equal gaps between all columns: class='row g-4'. Use gx-4 for horizontal gaps only, gy-3 for vertical gaps only. This is cleaner than adding margins to individual columns.

04 Demo — 2-Column Profile + Link List Layout

Build the core layout of the Social Bio Link App: a left column for the profile image and a right column for the name, role, and social link list. On mobile they stack vertically; on medium screens and up they sit side by side. This two-column responsive pattern is the foundation of nearly every profile page on the web — Naukri recruiter cards, LinkedIn profiles, GitHub user pages.

```
<div class='container py-5'>
  <div class='row align-items-center g-4'>
```

Page wrapper with vertical padding

Row — vertically centred children, 4-unit gap

```

    <!-- Left column: Profile image -->
    <div class='col-12 col-md-4 text-center'>      Full width on mobile, 33% on md+
    <img src='profile.jpg' alt='Profile photo'
    class='img-fluid rounded-circle' style='max-width:180px;'>
    img-fluid = responsive; rounded-circle = circular
    </div>

    <!-- Right column: Name, role, links -->
    <div class='col-12 col-md-8'>                  Full width on mobile, 66% on md+
        <h2>Riya Patel</h2>                        Name
    <p class='text-muted'>Web Developer · Ahmedabad</p>
    Role line — text-muted is a Bootstrap utility
    <ul class='list-group list-group-flush'>
    Bootstrap list — flush removes outer border
    <li class='list-group-item'><a href='#'>Instagram</a></li>
    Link item
    <li class='list-group-item'><a href='#'>LinkedIn</a></li>
    <li class='list-group-item'><a href='#'>GitHub</a></li>
    </ul>
    </div>
    </div>
</div>

```

TRY THIS LIVE — Build the Profile Layout

Create a new index.html with the Bootstrap CDN links. Paste the code above. Open in Chrome — the image and links should stack vertically. Open DevTools -> Device Simulation -> select iPhone SE. Confirm they stack. Switch to iPad Air. Confirm they sit side by side. Then resize slowly and watch the layout switch at exactly 768px.

WATCH — VIDEO REFERENCE (HINDI / ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Internet Technologies — lecture on CSS Frameworks and Bootstrap (IIT Kharagpur). Trainer: verify lecture number at nptel.ac.in before class.

Link — <https://nptel.ac.in/courses/106105084>

WHERE TO GO NEXT

You can now scaffold any responsive multi-column layout in minutes using Bootstrap Grid — without writing a single media query by hand.

Explore deeper — Read the official Bootstrap 5 Grid documentation at getbootstrap.com/docs/5.3/layout/grid — it is exceptionally well-written and includes live interactive

examples.

Try on your own — Recreate the TOPS Technologies course listing page layout using only Bootstrap Grid classes. Count the columns, identify the breakpoints, and match the layout as closely as possible.

Connect the dots — Session 10 adds Bootstrap's pre-built components — navbars, cards, buttons, and forms — the building blocks of every Bootstrap-based web app.

Going further — Look up Bootstrap's CSS Grid integration (display: grid inside Bootstrap 5.3) — it is a newer alternative to the traditional row/col system that gives you full CSS Grid power with Bootstrap's utility ecosystem.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a web development teacher. Explain Bootstrap's 12-column grid system in simple Hinglish using the analogy of dividing a thali into equal portions. Explain what col-12, col-6, and col-4 mean visually, and what the responsive prefix col-md-6 does. Keep it under 150 words.

SESSION

10

60 MIN · CSS FRAMEWORK

Bootstrap Components

◆ Learning Objectives

- Build a responsive navbar with a brand, nav links, and mobile hamburger toggle
- Apply Bootstrap button classes with colour variants, outline styles, and sizes
- Construct card components with images, headings, body text, and action buttons
- Style forms with Bootstrap's form-label, form-control, and form-select classes
- Build the Social Bio Link App navbar using Bootstrap Navbar component

01 Navbar — Responsive Navigation

The Bootstrap Navbar is the most complex component you will use in this module — it automatically collapses into a hamburger menu on small screens and expands on larger ones. It requires both the CSS and JS bundle to work correctly. Understanding its structure is important because virtually every Bootstrap-based web app in India, from fintech dashboards to ed-tech platforms, uses this exact component.

```
<nav class='navbar navbar-expand-lg bg-body-tertiary'>
  navbar-expand-lg: collapses below 992px
  <div class='container'>                                Constrains navbar content to page width
  <a class='navbar-brand' href='#'>BioLink</a>
    Brand/logo — always visible
  <button class='navbar-toggler' type='button'
    Hamburger button — visible only on mobile
    data-bs-toggle='collapse'                                Tells Bootstrap JS what to do on click
    data-bs-target='#navMain'                                ID of the collapsible div below
  <span class='navbar-toggler-icon'></span>
    Bootstrap draws the icon via CSS
  </button>
  <div class='collapse navbar-collapse' id='navMain'>
    The collapsible section — id must match data-bs-target
  <ul class='navbar-nav ms-auto mb-2 mb-lg-0'>
    ms-auto pushes links to the right
  <li class='nav-item'><a class='nav-link active' href='#'>Home</a></li>
    active highlights the current page
  <li class='nav-item'><a class='nav-link' href='#'>My Links</a></li>
  <li class='nav-item'><a class='nav-link' href='#'>Contact</a></li>
  </ul>
  </div>
</div>
</nav>
```

CONCEPT — How the Hamburger Toggle Works

On screens below 992px, Bootstrap hides the nav links div using `display: none`. When the hamburger button is clicked, Bootstrap's JavaScript toggles the 'show' class on the target div (`id='navMain'`), which changes it to `display: block`. The `data-bs-toggle` and `data-bs-target` attributes on the button tell Bootstrap JS exactly which div to toggle. No custom JavaScript is needed — it is all handled by the Bootstrap bundle.

TRY THIS LIVE — Build the Bio Link Navbar

Add the navbar code to your project's `index.html`, above the container. Open in the browser. Resize the window below 992px (or use DevTools device simulation) — the links should disappear and the hamburger icon should appear. Click the icon — the links should drop down. Confirm the JS bundle script tag is at the bottom of body if this does not work.

02 Buttons

Bootstrap buttons use a two-class system: `btn` sets the base styling and `btn-{variant}` sets the colour. Every variant also has an outline version that shows only the border with no fill, ideal for secondary actions. Buttons can be applied to button tags, input elements, and anchor tags — and they all look identical.

```
<!-- Filled variants -->
<button class='btn btn-primary'>Primary</button>
  Blue — main call to action
<button class='btn btn-secondary'>Secondary</button>
  Grey — secondary action
<button class='btn btn-success'>Success</button>
  Green — confirm, save, submit
<button class='btn btn-danger'>Danger</button>
  Red — delete, remove, error action
<button class='btn btn-dark'>Dark</button>      Black — often used for social link buttons
<!-- Outline variants -->
<button class='btn btn-outline-primary'>Outline</button>
  Border only, no fill
<button class='btn btn-outline-dark'>Dark outline</button>
  Common for ghost buttons
<!-- Sizes -->
<button class='btn btn-primary btn-sm'>Small</button>
  Smaller than default
<button class='btn btn-primary btn-lg'>Large</button>
  Larger than default
<button class='btn btn-primary w-100'>Full width</button>
  Stretches to container width — common for mobile CTAs
```

03 Cards

The Bootstrap card is the most universally used component on the web — every course listing, product tile, social link preview, and blog post thumbnail uses this pattern. A card is a white box with a border and optional image, title, text, and footer. You place cards inside Bootstrap Grid columns to create responsive card grids.

```
<div class='card'>
  <img src='thumbnail.jpg' class='card-img-top' alt='Course cover'>
    Image sits flush against the top of the card
  <div class='card-body'>
    <h5 class='card-title'>Web Development</h5>
    Card heading
    <p class='card-text'>Master HTML, CSS & Bootstrap in 12 sessions.</p>
    Body text
    <a href='#' class='btn btn-primary'>Enroll Now</a>
    Call-to-action button inside the card
  </div>
</div>

<!-- Card in a responsive grid -->
<div class='row g-4'>
  <div class='col-12 col-md-6 col-lg-4'>
    <div class='card h-100'>...</div>
  </div>
</div>
```

White box with border — card container

Padded content area

Row with 4-unit gutters between cards

Card takes full -> half -> third width as screen grows

h-100 makes all cards the same height in a row

PRO TIP — h-100 for Equal-Height Cards

When placing Bootstrap cards in a grid, cards in the same row will be different heights depending on their content. Adding h-100 to the card div makes every card stretch to the full height of its column, creating a visually uniform grid. This is a near-universal convention in Bootstrap card grids — always add it.

04 Forms, Carousel & More

Bootstrap gives every form element a polished, consistent style with three classes: form-label on the label tag, form-control on text and email inputs, and form-select on dropdowns. Each field should be wrapped in a div with mb-3 (margin bottom) to create spacing between fields. This is the same pattern used in every CRUD form in every Bootstrap application.

```
<form>
  <div class='mb-3'>
    Margin bottom 3 — spacing between each field block
```



```
<label for='fullname' class='form-label'>Full Name</label>
  form-label styles the label
<input type='text' class='form-control' id='fullname'>
  form-control styles the input — full width, rounded border
</div>
<div class='mb-3'>
<label for='platform' class='form-label'>Platform</label>
<select class='form-select' id='platform'>
  form-select styles the dropdown
    <option value=''>-- Choose --</option>
    <option value='ig'>Instagram</option>
    <option value='li'>LinkedIn</option>
  </select>
</div>
<button type='submit' class='btn btn-primary w-100'>Add Link</button>
  Full-width submit button
</form>
```

Component	Key Classes	JS Required?
Navbar	navbar, navbar-expand-lg, navbar-brand, navbar-toggler, collapse navbar-collapse, nav-item, nav-link	Yes — for hamburger toggle
Button	btn, btn-{variant}, btn-outline-{variant}, btn-sm, btn-lg, w-100	No
Card	card, card-img-top, card-body, card-title, card-text, h-100	No
Form	form-label, form-control, form-select, mb-3	No
Carousel	carousel, carousel-inner, carousel-item, carousel-control-prev, carousel-control-next	Yes — for slide transitions
Modal	modal, modal-dialog, modal-content, modal-header, modal-body, modal-footer	Yes — for open/close

TRY THIS LIVE — Bio Link App Shell

Combine what you have built: paste the Navbar at the top of body, add a container below it with the 2-column profile layout from Session 9 (image + link list), then add a row of 3 Bootstrap cards below the profile — each card representing one social link with a title, short description, and 'Visit' button. This is 80% of the finished

Mini Project structure.

WATCH — VIDEO REFERENCE (HINDI / ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Internet Technologies — lecture on UI Component Libraries (IIT Kharagpur). Trainer: verify lecture number at nptel.ac.in before class.

Link — <https://nptel.ac.in/courses/106105084>

WHERE TO GO NEXT

You can now build complete, interactive page shells using Bootstrap components — the skill that lets you prototype a full UI in under an hour.

Explore deeper — Browse every component in the Bootstrap 5 documentation at getbootstrap.com/docs/5.3/components — each has a live demo and a copyable code example.

Try on your own — Pick any Indian app's landing page — Razorpay, Zerodha, or CoinDCX — and identify which Bootstrap components (or custom equivalents) they use in their hero section and navbar.

Connect the dots — Session 11 teaches Bootstrap utility classes — the spacing, typography, and colour shortcuts that let you customise every component and override Bootstrap's defaults with clean, minimal CSS.

Going further — Look up Bootstrap Icons (icons.getbootstrap.com) — over 2,000 free SVG icons that integrate directly with Bootstrap components. Adding an icon to a button is one class.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Bootstrap teacher. Explain the Bootstrap Navbar hamburger menu in simple Hinglish — what happens on mobile, what `data-bs-toggle` and `data-bs-target` do, and why the JS bundle script tag is needed. Use the analogy of a folded menu at a restaurant that opens when you tap it. Keep it under 150 words.

SESSION

11

60 MIN · CSS FRAMEWORK

Bootstrap Utilities & Customisation

◆ Learning Objectives

- Apply Bootstrap spacing utilities for margin and padding without writing custom CSS
- Style text using Bootstrap typography utilities for weight, size, colour, and alignment
- Use colour utilities for text and background colouring with opacity control
- Show and hide elements responsively using Bootstrap display utilities
- Override Bootstrap styles with custom CSS loaded after the Bootstrap stylesheet
- Build a Bootstrap 5 dark mode toggle using the data-bs-theme attribute

01 Spacing Utilities — Margin & Padding Without Custom CSS

Bootstrap includes a complete set of spacing utilities that let you set margin and padding on any element using a class, without touching your CSS file at all. The pattern is m or p (margin or padding), followed by a direction shorthand (t, b, s, e, x, y, or blank for all sides), followed by a value from 0 to 5. These utilities cover 80% of the spacing you will ever need to add to a Bootstrap layout.

<code>m-0</code>	<code>-> margin: 0 (remove all margin)</code>	0 = 0, 1 = 0.25rem, 2 = 0.5rem, 3 = 1rem, 4 = 1.5rem, 5 = 3rem
<code>m-3</code>	<code>-> margin: 1rem on all sides</code>	Most commonly used margin class
<code>mx-auto</code>	<code>-> margin-left: auto; margin-right: auto</code>	
	Centres a block element horizontally	
<code>mt-4</code>	<code>-> margin-top: 1.5rem</code>	t = top, b = bottom, s = start (left), e = end (right)
<code>mb-2</code>	<code>-> margin-bottom: 0.5rem</code>	
<code>ms-auto</code>	<code>-> margin-start: auto</code>	Pushes element to the right — used in navbars
<code>p-4</code>	<code>-> padding: 1.5rem on all sides</code>	p = padding; same size scale as margin
<code>py-5</code>	<code>-> padding-top: 3rem; padding-bottom: 3rem</code>	
	y = vertical (top + bottom)	
<code>px-3</code>	<code>-> padding-left: 1rem; padding-right: 1rem</code>	
	x = horizontal (left + right)	
<code>p-0</code>	<code>-> padding: 0</code>	Remove default padding — useful on images inside cards

PRO TIP — Responsive Spacing

Bootstrap spacing utilities accept breakpoint prefixes just like columns. `mb-4 mb-md-0` means: margin-bottom 4 on mobile (so stacked items have space), margin-bottom 0 on md+ (because they're side by side and don't need it). This pattern appears constantly in Bootstrap layouts — you saw it in the profile card demo in Session 9.

02 Typography & Colour Utilities

Bootstrap's typography utilities let you set font weight, size, alignment, and text colour with a single class. Combined with colour utilities for backgrounds, these classes eliminate the need for custom CSS in the majority of text-styling tasks. They are especially useful for quick prototyping and for creating consistent visual hierarchies without opening a CSS file.

```
/* Typography utilities */
fw-bold      -> font-weight: bold (700)
fs-1         -> font-size: 2.5rem
text-center  -> text-align: center
text-uppercase -> text-transform: uppercase
text-muted   -> muted grey text
text-truncate -> overflow: hidden; text-overflow: ellipsis
    Truncates long text with ... — requires fixed width
lh-1         -> line-height: 1
/* Colour utilities */
text-primary -> Bootstrap blue text
bg-primary   -> Bootstrap blue background
bg-opacity-25 -> background at 25% opacity
```

Also: fw-normal (400), fw-semibold (600), fw-light (300)
fs-1 (largest) to fs-6 (1rem) — mirrors heading sizes
Also: text-start, text-end
Also: text-lowercase, text-capitalize
Bootstrap's go-to for secondary/helper text
Also: lh-sm (1.25), lh-base (1.5), lh-lg (2)
Also: text-success (green), text-danger (red), text-dark, text-white
Also: bg-light, bg-dark, bg-success, bg-danger, bg-transparent
Use with bg-{colour} — also 50, 75

Utility Category	Class Pattern	Common Use Case
Margin	m{direction}-{0–5}	Space between sections, cards, headings
Padding	p{direction}-{0–5}	Inner breathing room in containers, cards, navbars
Font weight	fw-{bold/semibold/normal/light}	Highlight names, headings, labels
Font size	fs-{1–6}	Hierarchy without custom font-size rules
Text colour	text-{variant}	Muted helper text, status messages, labels
Background	bg-{variant}	Section backgrounds, badges, alerts

03 Display Utilities & Responsive Visibility

Bootstrap's display utilities are shorthand for CSS display properties — but crucially, they accept breakpoint prefixes. This lets you show and hide elements at specific screen widths purely with HTML classes, without writing a single media query. This pattern is used constantly for navigation: show a full menu on desktop, hide it on mobile where the hamburger toggle takes over.

```
d-none          -> display: none (hidden everywhere)
  Starting point for elements you want to show conditionally
d-md-block      -> display: block from md (768px) and up
  Hidden on mobile, visible on tablet+
d-none d-md-flex -> hidden on mobile, flex container on tablet+
  Common pattern for desktop nav bars and desktop sidebars
d-md-none       -> display: none from md and up (visible only on mobile)
  Show only on mobile — e.g. mobile-only banner or CTA
d-flex          -> display: flex (Flexbox container)
  Enables Bootstrap's Flexbox utilities on this element
justify-content-between -> justify-content: space-between
  Pushes children to the edges of the flex container
justify-content-center  -> justify-content: center
  Centres children horizontally
align-items-center      -> align-items: center
  Vertically centres children in a flex row
gap-3                   -> gap: 1rem between all flex/grid children
  Cleaner than adding margin to each child
```

CONCEPT — The d-none + d-{bp}-{value} Pattern

The standard Bootstrap pattern for responsive visibility is: start with d-none (hidden everywhere), then add d-{breakpoint}-{display} to show it from that breakpoint up. For example, d-none d-lg-flex means the element is hidden on mobile and tablet, then becomes a flex container from large screens. This replaces writing @media (min-width: 992px) { display: flex; } in your CSS file.

04 Customising Bootstrap — Your CSS on Top

Bootstrap is designed to be overridden. The rule is simple: load Bootstrap's CSS first, then load your custom CSS after. Because CSS cascades, your rules win over Bootstrap's when they have the same or higher specificity. For brand colours, custom fonts, and small layout tweaks, this two-file pattern — bootstrap.min.css then custom.css — is all you need.

```

<!-- In <head> – order matters -->
<link rel='stylesheet'
href='https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css'>
    Bootstrap first — establishes all defaults
<link rel='stylesheet' href='custom.css'>    Your CSS second — your rules override Bootstrap's
/* custom.css – override examples */
.btn-primary { background-color: #E34C26; border-color: #E34C26; }
    Change the primary button colour to TOPS orange
.navbar { font-family: 'Segoe UI', sans-serif; }
    Custom font on the navbar
.card { border-radius: 16px; border: none; box-shadow: 0 4px 12px rgba(0,0,0,0.07); }
    Softer cards — removes border, adds shadow
:root { --bs-primary: #E34C26; }
    Override Bootstrap CSS variable — cascades through
    all primary-coloured components

```

WARNING — Never Edit bootstrap.min.css Directly

Never modify the Bootstrap CSS file itself. When Bootstrap releases a bug fix or update, you would need to re-apply every edit manually. Always override in your own separate file loaded after Bootstrap. If you need to deeply customise Bootstrap colours, font sizes, and spacing, look up 'Bootstrap 5 Sass customisation' — it compiles a custom Bootstrap build with your variables baked in.

TRY THIS LIVE — Bootstrap Dark Mode Toggle

Bootstrap 5.3 has built-in dark mode. Add `data-bs-theme='light'` to your html tag. Add a button with `id='themeBtn'`. Paste this JavaScript before `</body>`: `const btn = document.getElementById('themeBtn');`
`btn.addEventListener('click', () => { const html = document.documentElement; const isDark =`
`html.getAttribute('data-bs-theme') === 'dark'; html.setAttribute('data-bs-theme', isDark ? 'light' : 'dark');`
`btn.textContent = isDark ? '■ Dark Mode' : 'Light Mode'; });` Every Bootstrap component — navbar, cards, buttons, forms — switches colour automatically. No custom CSS needed.

WATCH — VIDEO REFERENCE (HINDI / ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Internet Technologies — lecture on CSS Utilities and Design Systems (IIT Kharagpur). Trainer: verify lecture number at nptel.ac.in before class.

Link — <https://nptel.ac.in/courses/106105084>

WHERE TO GO NEXT

You can now style any Bootstrap component using utilities and override it cleanly with custom CSS — the foundation of every production Bootstrap project.

Explore deeper — Read Bootstrap's Utility API documentation at getbootstrap.com/docs/5.3/utilities/api — it shows how to generate your own utility classes using the same pattern Bootstrap uses internally.

Try on your own — Take your Bio Link App shell and apply at least 10 Bootstrap utility classes to tighten spacing, set font weights, and adjust colours — without writing a single new line of custom CSS.

Connect the dots — Session 12 is the Integration Review — you combine everything from Sessions 1–11 (semantic HTML, CSS, responsive design, Bootstrap grid, components, and utilities) into the complete Bio Link landing page.

Going further — Look up Bootstrap 5's CSS Custom Properties (`--bs-primary`, `--bs-font-sans-serif`, `--bs-border-radius`) — overriding these variables in `:root` changes Bootstrap's entire design language in one block of CSS.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Bootstrap teacher. Explain Bootstrap utility classes in simple Hinglish using the analogy of ready-made masala packets — instead of grinding individual spices (writing custom CSS), you use a pre-made mix (utility class) for common flavours. Give three examples of spacing utilities and explain what problem they replace. Keep it under 150 words.

SESSION

12

60 MIN · INTEGRATION

Integration Review

◆ Learning Objectives

- Explain the three-layer architecture of a Bootstrap project: HTML structure, Bootstrap classes, custom CSS
- Load Bootstrap CSS before custom CSS and explain why the order matters
- Build a complete, responsive Bio Link landing page from a blank file
- Use semantic HTML5 tags as the structural skeleton that Bootstrap classes style
- Verify a page's responsiveness at 375px, 768px, and 1280px using DevTools
- Identify the correct tool (HTML / Bootstrap utility / custom CSS) for each styling task

01 The Three-Layer Architecture

Every professional Bootstrap project has exactly three layers. The first is semantic HTML — the structural skeleton using the tags you learned in Sessions 1–4. The second is Bootstrap classes — grid, components, and utilities applied directly in the HTML. The third is custom CSS — brand colours, custom fonts, and component overrides loaded after Bootstrap. Each layer has a clear job, and they never fight when kept in the right order.

What NOT to Do

Mix inline styles with Bootstrap classes, load custom CSS before Bootstrap, override Bootstrap with !important on every rule, use div soup instead of semantic tags, write media queries for things Bootstrap utilities already handle. Each of these creates maintenance debt that doubles the time to make simple changes.

What TO Do

Semantic HTML5 for structure -> Bootstrap CDN -> your custom CSS (in that exact order). Let Bootstrap handle grid, spacing, components, and responsive visibility. Reserve custom CSS for brand colours, custom fonts, box shadows, and anything Bootstrap's utilities cannot do alone.

CONCEPT — Which Tool for Which Job?

Before writing any CSS, ask: can Bootstrap already do this? Spacing, colour, typography, display, flex alignment — Bootstrap utilities cover all of these. If Bootstrap can do it, use a class. If you need a brand colour, a custom font, a specific box shadow, or an override to Bootstrap's defaults — write custom CSS. If it is structural (what is this element?) — it is HTML semantics.

Task	Right Tool	Wrong Approach
Add 1.5rem top margin to a heading	mt-4 Bootstrap utility class	margin-top: 1.5rem in custom CSS
Set the brand primary colour	Custom CSS — override --bs-primary variable	Editing bootstrap.min.css directly

Task	Right Tool	Wrong Approach
Create a 3-column responsive grid	col-12 col-md-4 Bootstrap Grid	Writing a custom CSS Grid with 3 media queries
Label a page's navigation area semantically	nav HTML5 tag	div class='navigation'
Hide a sidebar on mobile	d-none d-md-block Bootstrap utility	@media (max-width: 767px) { display: none; }
Custom card border radius	Custom CSS — .card { border-radius: 16px; }	Adding inline style='border-radius:16px'

02 The Full Page Structure

Before writing a single class, sketch the page's semantic outline: what is the header, what is the main content, which sections does main contain, and what is the footer? This outline becomes your HTML skeleton — Bootstrap classes and custom styles are layered on top. When you build in this order, every element has a clear identity before it has a style.

```
<!DOCTYPE html>
<html lang='en' data-bs-theme='light'>                                data-bs-theme enables Bootstrap dark mode toggle
<head>
  <meta charset='UTF-8'>
  <meta name='viewport' content='width=device-width, initial-scale=1.0'>
    Required for responsive design
  <title>Bio Link — Riya Patel</title>
  <link rel='stylesheet'
href='https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css'>
    Layer 2: Bootstrap — always before custom
  <link rel='stylesheet' href='custom.css'>                                Layer 3: Custom CSS — always after Bootstrap
</head>
<body>
<header> <!-- Bootstrap Navbar --> </header>
  Layer 1: Semantic HTML — Bootstrap classes go inside
  <main>
    <section class='hero py-5 text-center'> <!-- Profile --> </section>
      Semantic section + Bootstrap utilities
    <section class='links py-4'> <!-- Link cards --> </section>
  </main>
  <footer class='py-4 text-center text-muted'> <!-- Copyright --> </footer>
```

```
<script
src='https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js'></s
cript>
  Bootstrap JS — always last before </body>
</body></html>
```

03 Demo — Static Bio Link Landing Page

Build the complete static Bio Link page combining all three layers. The demo uses the navbar from Session 10, the responsive grid from Session 9, Bootstrap cards for link previews, and a custom.css file for brand colours and card styling. Complete this demo and the Mini Project becomes a matter of adding a working form — the structure is already done.

```
<!-- Hero section — inside <main> -->
<section class='hero py-5 text-center'>                                Full-width section with vertical padding
  <div class='container'>
    <img src='profile.jpg' alt='Profile photo'
    class='rounded-circle mb-3' style='width:120px;height:120px;object-fit:cover;'>
      object-fit:cover keeps image ratio inside the circle
    <h1 class='fs-3 fw-bold mb-1'>Riya Patel</h1>
      Name — Bootstrap typography utilities
    <p class='text-muted mb-0'>Web Developer · Ahmedabad</p>
      Role — muted colour utility
    </div>
  </section>
<!-- Link cards section -->
<section class='links py-4'>
  <div class='container'>
    <div class='row g-3 justify-content-center'>
      Centred grid with 3-unit gutters
    <div class='col-12 col-sm-10 col-md-8 col-lg-6'>
      Card narrows as screen grows — centered stacked layout
    <a href='#' class='card link-card text-decoration-none mb-2'>
      Anchor wrapping the card — removes underline
    <div class='card-body d-flex align-items-center gap-3'>
      Flex row — icon and text side by side
    <img src='instagram.svg' alt='Instagram' width='32' height='32'>
      Platform icon
    <span class='fw-semibold'>Instagram</span>
      Platform name
    </div>
  </a>
  Repeat this card block for each social link
</div>
```

```
</div>
</div>
</section>
```

```
/* custom.css — brand layer on top of Bootstrap */
:root { --bs-primary: #E34C26; }           Override Bootstrap primary colour globally
body { font-family: 'Segoe UI', system-ui, sans-serif; }
    Custom font stack for the whole page
.hero { background: linear-gradient(160deg, #fff7f5 0%, #ffffff 100%); }
    Subtle warm gradient behind the profile
.link-card {                               Custom card style for each link item
    border-radius: 14px;                   Rounder corners than Bootstrap default
    border: 1.5px solid #f0e8e5;           Subtle brand-tinted border
    transition: transform 0.15s ease, box-shadow 0.15s ease;
        Smooth hover animation
    color: #1a1a1a; }                     Dark text on the card
.link-card:hover {                         Hover effect
    transform: translateY(-2px);           Lifts card slightly on hover
    box-shadow: 0 6px 20px rgba(227,76,38,0.12); }
    Brand-tinted shadow on hover
```

TRY THIS LIVE — Assemble the Complete Page

Build the full page in one file: HTML boilerplate -> Bootstrap CDN -> custom.css link -> Navbar from Session 10 -> Hero section (profile photo + name) -> Link cards section (3–5 cards) -> Footer -> Bootstrap JS bundle. Open in Chrome. Use DevTools to verify at iPhone SE (375px), iPad Air (768px), and a full desktop view. This is exactly what you submit for the Mini Project — the form is the only remaining addition.

04 Responsive QA Checklist

Before calling any page complete, run through this checklist at three widths in Chrome DevTools Device Simulation. These are the same checks a front-end developer performs before pushing code to production. Missing any one of them is a common reason student projects lose marks in reviews.

Check	How to Verify	Session It Came From
Viewport meta tag present	Resize to 375px — page should fill screen width, not zoom out	Session 8
Bootstrap CSS loads before custom	DevTools Network tab — bootstrap.min.css before custom.css	Session 9 / 11

Check	How to Verify	Session It Came From
All images have alt attributes	DevTools Accessibility panel — no 'Image has no alt' errors	Session 2
Semantic landmark tags used	Elements panel — header, nav, main, section, footer visible	Session 4
Layout at 375px (iPhone SE)	No horizontal scroll; text readable without zooming; single column	Session 8
Layout at 768px (iPad Air)	2-column or wider layout appears; navbar still usable	Session 9
Layout at 1280px (desktop)	Full layout; max-width container centred; no broken elements	Session 8
Navbar hamburger works on mobile	Click hamburger at <992px — links drop down and close correctly	Session 10
Card hover effect visible	Hover over each link card — it lifts and shows shadow	Session 12
Dark mode toggle works	Click toggle — all Bootstrap components switch colour	Session 11

“A web page is not finished when everything is added. It is finished when nothing can be taken away.”

WATCH — VIDEO REFERENCE (HINDI / ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Internet Technologies — lecture on Full-Stack Web Development Integration (IIT Kharagpur).
Trainer: verify lecture number at nptel.ac.in before class.

Link — <https://nptel.ac.in/courses/106105084>

WHERE TO GO NEXT

You have completed the full HTML and CSS layer of front-end development — semantic structure, custom styling, responsive layout, and a production-grade Bootstrap project.

Explore deeper — Read through Google's web.dev 'Learn HTML' and 'Learn CSS' courses (web.dev/learn) — they are the most current, comprehensive references and align precisely with what was covered in this module.

Try on your own — Open three real bio link pages — linktr.ee, bio.link, or a creator's actual page — and reverse-engineer their HTML structure using DevTools. Identify every Bootstrap-equivalent pattern you recognise.

Connect the dots — The Mini Project brings everything together: add the form-to-card feature so users can input a URL and platform name, and a styled card appears on the page.

Going further — Look up CSS animations and transitions (animation, @keyframes, transition) to add micro-interactions — the hover effects on the link cards in today's demo are a starting point.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a front-end development teacher. A student asks: 'When should I use a Bootstrap utility class versus writing my own CSS?' Give a clear rule of thumb in simple Hinglish with three examples — one that should always use a utility class, one that should always use custom CSS, and one that could use either. Keep it under 150 words.

REVIEW



INTERVIEW PREP · MODULE 2

Interview Questions

◆ SE – Introduction to Web Technologies: HTML and CSS

Rehearse each answer out loud, aiming for about 60 seconds. The hint is only a nudge — try answering first, then compare with the model answer below.

1

What is the difference between front-end and back-end in web development?

Think about what the user sees versus what happens on the server.

MODEL ANSWER

Front-end refers to everything the user sees and interacts with in the browser — HTML structure, CSS styling, and JavaScript behaviour. Back-end refers to the server, application logic, and database that run behind the scenes. When you click a login button, the front-end collects your credentials and the back-end verifies them against the database and returns a response. A full-stack developer works on both sides.

2

Explain what happens when a browser renders an HTML page.

Consider the steps from downloading the HTML file to displaying it on screen.

MODEL ANSWER

When you request a URL, the browser downloads the HTML file from the server. It parses the HTML top-to-bottom, building the Document Object Model (DOM) — a tree structure of all the elements. When it encounters linked CSS files, it downloads and parses them to build the CSSOM (CSS Object Model). The browser combines the DOM and CSSOM into a render tree, calculates layout (size and position of each element), and then paints pixels to the screen.

3

What is the DOM, and why is it important in web development?

Focus on how browsers represent and interact with the structure of a webpage.

MODEL ANSWER

The DOM (Document Object Model) is the browser's in-memory tree representation of an HTML page, where each HTML tag becomes a node. It is important because JavaScript uses the DOM to read and modify page content dynamically without reloading the page. For example, when you type in a search box and suggestions appear, JavaScript is querying and updating the DOM in real time. Understanding the DOM is fundamental for building interactive web applications.

4

What is the purpose of the `<!DOCTYPE html>` declaration at the start of an HTML file?

Consider how browsers interpret the HTML code.

MODEL ANSWER

The `<!DOCTYPE html>` declaration tells the browser that this document follows the HTML5 standard. Without it, browsers may enter 'quirks mode' and try to render the page as if it were old HTML, which can cause inconsistent layout and styling across different browsers. It is not a tag or element — it is an instruction to the browser. It should always be the very first line of any HTML document.

5

List the basic structure of an HTML document, naming the main tags used.

MODEL ANSWER

Every HTML document starts with `<!DOCTYPE html>`, followed by `<html>` as the root element. Inside `<html>` there are two sections: `<head>`, which contains metadata like the page title, character set, and links to CSS, and `<body>`, which contains all the visible content — headings, paragraphs, images, and links. This structure is the skeleton every web page is built on.

6

Which HTML tags would you use to create a bulleted list and a numbered list?

There are two different tags for these list types.

MODEL ANSWER

A bulleted (unordered) list uses the `<ul>` tag, and each item inside uses `<li>`. A numbered (ordered) list uses the `<ol>` tag with `<li>` items as well. The browser automatically adds bullet points for `<ul>` and sequential numbers for `<ol>`. You can nest lists inside each other to create multi-level lists.

7

What is the purpose of the `<form>` tag in HTML?

Think about user input and data submission.

MODEL ANSWER

The `<form>` tag groups input elements together and defines how user-submitted data is sent to a server. It has two key attributes: `action` (the URL the data is sent to) and `method` (either GET or POST). GET appends data to the URL and is used for searches; POST sends data in the request body and is used for sensitive data like passwords. Without a form tag, inputs exist but have no built-in mechanism to be submitted.

8

Name three different types of input fields you can use inside an HTML form.

MODEL ANSWER

A `<input type='text'>` collects a single line of plain text. A `<input type='email'>` collects an email address and automatically validates the format. A `<textarea>` collects multi-line text, useful for messages or descriptions. Other types include `<input type='password'>`, `<input type='checkbox'>`, `<select>` for dropdown menus, and `<input type='submit'>` for the submission button.

9

What are semantic HTML5 tags, and why should you use them?

Consider how these tags help browsers and assistive technologies understand page structure.

MODEL ANSWER

Semantic HTML5 tags are elements whose names describe their meaning and purpose on the page, rather than just their visual appearance. Tags like `<header>`, `<nav>`, `<article>`, `<section>`, and `<footer>` tell the browser and assistive technologies (screen readers) exactly what role that content plays. This improves accessibility for users with disabilities, helps search engines index content more accurately, and makes the code easier for developers to read and maintain.

10

Explain the difference between inline, internal, and external CSS.**MODEL ANSWER**

Inline CSS is written directly on an HTML element using the `style` attribute (e.g., `<p style='color:red'>`), affecting only that element. Internal CSS is written inside a `<style>` tag in the `<head>` section and applies styles to the current page only. External CSS is written in a separate `.css` file linked using `<link rel='stylesheet' href='style.css'>`, and can apply styles to many pages at once. External CSS is the preferred approach because it separates concerns and avoids repeating styles across pages.

11

What is the CSS box model, and what are its main components?*Think about how spacing and borders are applied to elements.***MODEL ANSWER**

The CSS box model describes how every HTML element is represented as a rectangular box with four layers. The innermost layer is the content (text, image, etc.), surrounded by padding (space inside the border), then the border itself, and finally the margin (space outside the border that separates the element from neighbours). Understanding the box model is essential for controlling layout and spacing — for example, `box-sizing: border-box` makes width include padding and border, simplifying layout calculations.

12

Explain the difference between 'position: relative' and 'position: absolute' in CSS.*Consider how each one affects the element's placement context.***MODEL ANSWER**

An element with `position: relative` stays in the normal document flow but can be nudged using `top/right/bottom/left` values relative to where it would normally sit. An element with `position: absolute` is removed from the normal document flow and positioned relative to its nearest ancestor that has a non-static position. This is commonly used to overlay elements on top of others, like a badge on a profile picture, where the card container is `relative` and the badge is `absolute`.

13

What is Flexbox in CSS, and when would you use it?*Think about arranging items in a row or column.***MODEL ANSWER**

Flexbox (Flexible Box Layout) is a CSS layout model that makes it easy to align and distribute items in one direction — either a row or a column. You apply `display: flex` to a container, then use properties like `justify-content` to align items along the main axis and `align-items` to align them along the cross axis. It is ideal for navigation bars, card rows, centering elements, and any layout where items need to be evenly spaced or dynamically sized. For two-dimensional layouts, CSS Grid is preferred.

14

What is a media query in CSS, and how does it help with responsive design?

Consider adapting layouts for different screen sizes.

MODEL ANSWER

A media query is a CSS rule that applies styles only when certain conditions are true, most commonly a specific screen width. For example, `@media (max-width: 768px) { ... }` applies styles only on screens 768px wide or narrower, like mobile phones. This allows a single HTML file to have different layouts for desktop, tablet, and mobile without needing separate pages. Media queries are the foundation of responsive web design.

15

Explain how the Bootstrap grid system works for creating responsive layouts.

MODEL ANSWER

Bootstrap's grid system divides the page into 12 equal columns. You place content inside `<div class='row'>` containers, then assign column classes to child elements (e.g., `col-md-6` means 'take 6 of 12 columns on medium screens'). On smaller screens, Bootstrap's breakpoints automatically stack columns vertically unless you specify otherwise. You wrap everything inside a `<div class='container'>` for centered, fixed-width layout or `container-fluid` for full-width. This system makes it fast to build consistent, responsive page structures.

16

Name two Bootstrap components you would use to build a navigation bar and a card.

MODEL ANSWER

For a navigation bar, Bootstrap provides the `<nav class='navbar'>` component with built-in support for a brand logo, collapsible links, and a hamburger menu on mobile. For a card, Bootstrap's `<div class='card'>` component provides a structured container with a card header, body, footer, and optional image — all styled and ready to use. Both components save significant development time and automatically work across different screen sizes.

17

Describe how you could override Bootstrap's default styles with your own CSS.

MODEL ANSWER

You link your custom CSS file after the Bootstrap CSS link in the `<head>` — this ensures your rules load last and can override Bootstrap's defaults. You then write more specific selectors or simply target the Bootstrap class with your own value, for example `.btn-primary { background-color: #ff5722; }`. Because CSS applies the last rule when specificity is equal, your custom file wins. Avoid modifying Bootstrap's source files directly, as that makes future updates difficult.

18

How would you integrate HTML, CSS, and Bootstrap together to create a static landing page mock-up?

MODEL ANSWER

Start with an HTML5 boilerplate and link Bootstrap's CSS in the `<head>` and its JS bundle before `</body>`. Use Bootstrap's grid and components (navbar, cards, buttons) to lay out the page structure quickly. Add your own `custom.css` file after Bootstrap's link to apply brand colours, fonts, and spacing overrides. Use semantic tags like `<header>`, `<main>`, and `<footer>` for structure, and test at multiple screen widths using Chrome DevTools' device toolbar to verify responsiveness.

19

How would you use browser developer tools to debug a layout issue where elements are overlapping?

MODEL ANSWER

Open DevTools with F12 and go to the Elements panel. Click the inspect cursor and hover over the overlapping elements to see their CSS box model (margin, padding, border, size) displayed visually. Check the Computed tab to see the final calculated values for position, width, and z-index, which often reveal why elements are stacking incorrectly. Common culprits are accidental `position: absolute` without a positioned parent, negative margins, or z-index conflicts. You can edit CSS values live in DevTools to test fixes before updating your code.

20

How could you use AI tools like ChatGPT or GitHub Copilot to help with HTML/CSS, and what would you do to ensure the suggestions are correct?

Think about how you would test or review AI-generated code before using it.

MODEL ANSWER

AI tools can quickly generate boilerplate HTML structures, suggest CSS properties, explain why a layout is breaking, or convert a rough description into working code. To verify suggestions, always paste the generated code into your project and test it in a browser across different screen sizes. Check that class names and attributes match your existing code, and use DevTools to confirm the visual output matches your intent. AI is a productivity tool, not an infallible authority — always review and test before committing.